$ vs .

```
ghci> :i ($)
($) :: (a -> b) -> a -> b
infixr 0 $
```

```
ghci> :i (.)
(.) :: (b -> c) -> (a -> b) -> a -> c
infixr 9 .
```

$$f\ \$\ x \equiv f\ x$$

```
f $ x ≡ f x
```

```
f $ g $ x ≡ f (g x)
```

```
f $ x ≡ f x
```

```
f $ g $ x ≡ f (g x)
```

```
f $ g y $ h $ x ≡ f (g y (h x))
```

```
ghci> :i ($)
($) :: (a -> b) -> a -> b
infixr 0 $
```

```
ghci> :i (.)
(.) :: (b -> c) -> (a -> b) -> a -> c
infixr 9 .
```

```
f . g ≡ \x -> f(g(x))
```

```
f . g ≡ \x -> f(g(x))
```

```
f . g . h ≡ f . (g . h)
```

```
f . g ≡ \x -> f(g(x))

f . g . h ≡ f . (g . h)

f . g y . h ≡ f . ((g y) . h)
```

```
f . g $ x ≡ (f . g) x
```

```
f . g $ x ≡ (f . g) x

f $ g . h ≡ f (g . h)
```

```
f . g $ x ≡ (f . g) x

f $ g . h ≡ f (g . h)

f . g $ h . l $ y ≡ (f . g) ((h . l) y)
```

```
f $ x
```

```
f :: Int -> b
x :: ?
f $ x :: ?
```

```
f $ x
```

```
f :: Int -> b
x :: Int
f $ x :: b
```

```
f . g
```

```
f :: String -> b
g :: ?
f . g :: ?
```

f . g

```
f :: String -> b
g :: a -> String
f . g :: a -> b
```

```
f $ g
```

```
f :: ?
g :: a -> String
f $ g :: Int
```

f $ g

```
f :: (a -> String) -> Int
g :: a -> String
f $ g :: Int
```

```
f $ g
```

```
f :: ?
g :: Int
f $ g :: ([a] -> a)
```

```
f $ g
```

```
f :: Int -> [a] -> a
g :: Int
f $ g :: ([a] -> a)
```

```
f 5 . g
```

```
f :: Int -> String -> Bool
g :: ?
f 5 . g :: ?
```

```
f 5 . g
```

```
f :: Int -> String -> Bool
g :: a -> String
f 5 . g :: a -> Bool
```

```
f 5 $ x
```

```
f :: Int -> String -> Bool
x :: ?
f 5 $ x :: ?
```

```
f 5 $ x
```

```
f :: Int -> String -> Bool
x :: String
f 5 $ x :: Bool
```

```
f 5 . g $ h “foo”
```

```
f :: ?
g :: ?
h :: ?
f 5 . g $ h “foo” :: ?
```

```
f 5 . g $ h “foo”
```

```
f :: Num d => d -> b -> c
g :: a -> b
h :: String -> a
f 5 . g $ h “foo” :: c
```

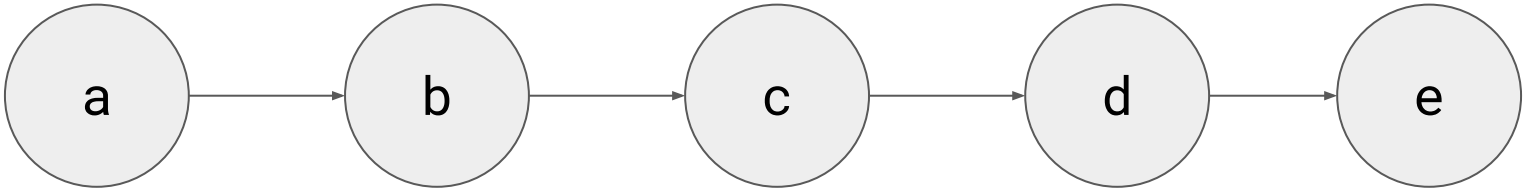
map

map f list

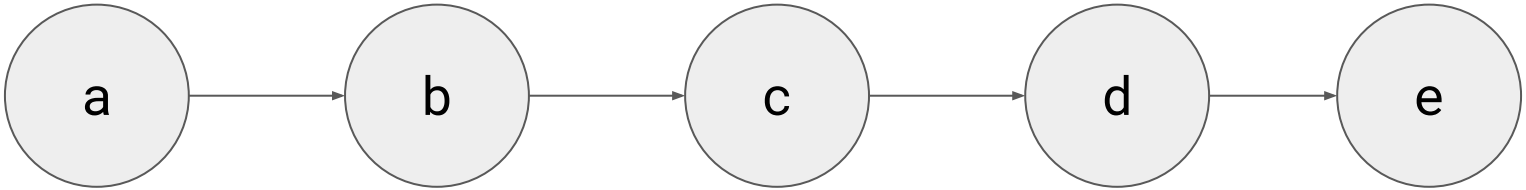
```
map :: (a -> b) -> [a] -> [b]

map _ [] = []
map f (x:xs) = f x : map f xs
```

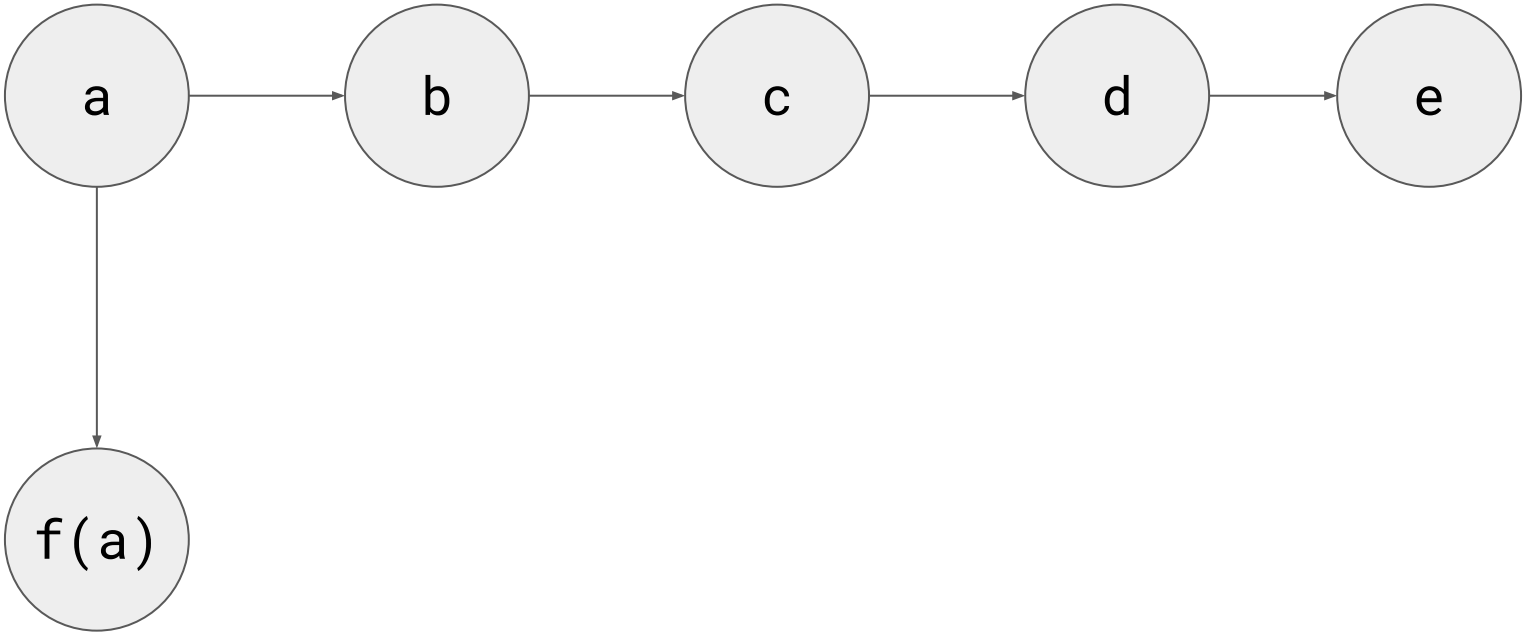
[a, b, c, d, e]



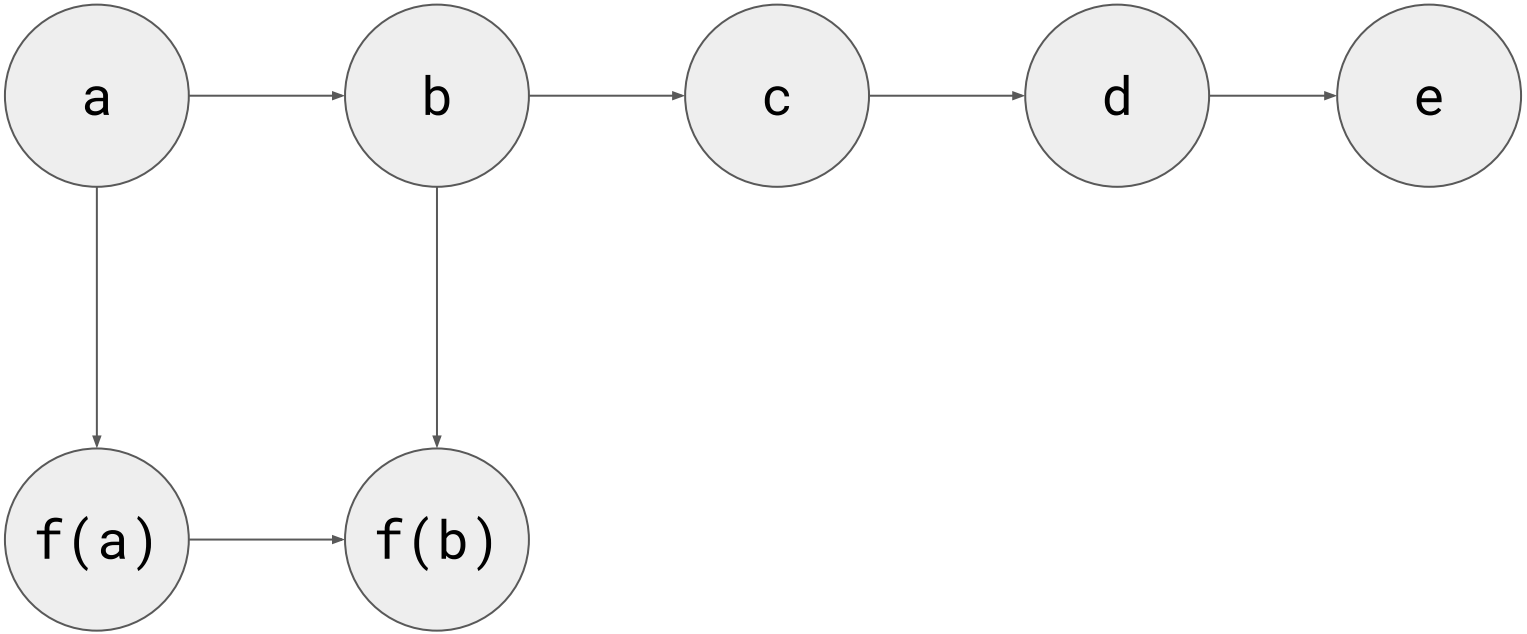
map f [a, b, c, d, e]



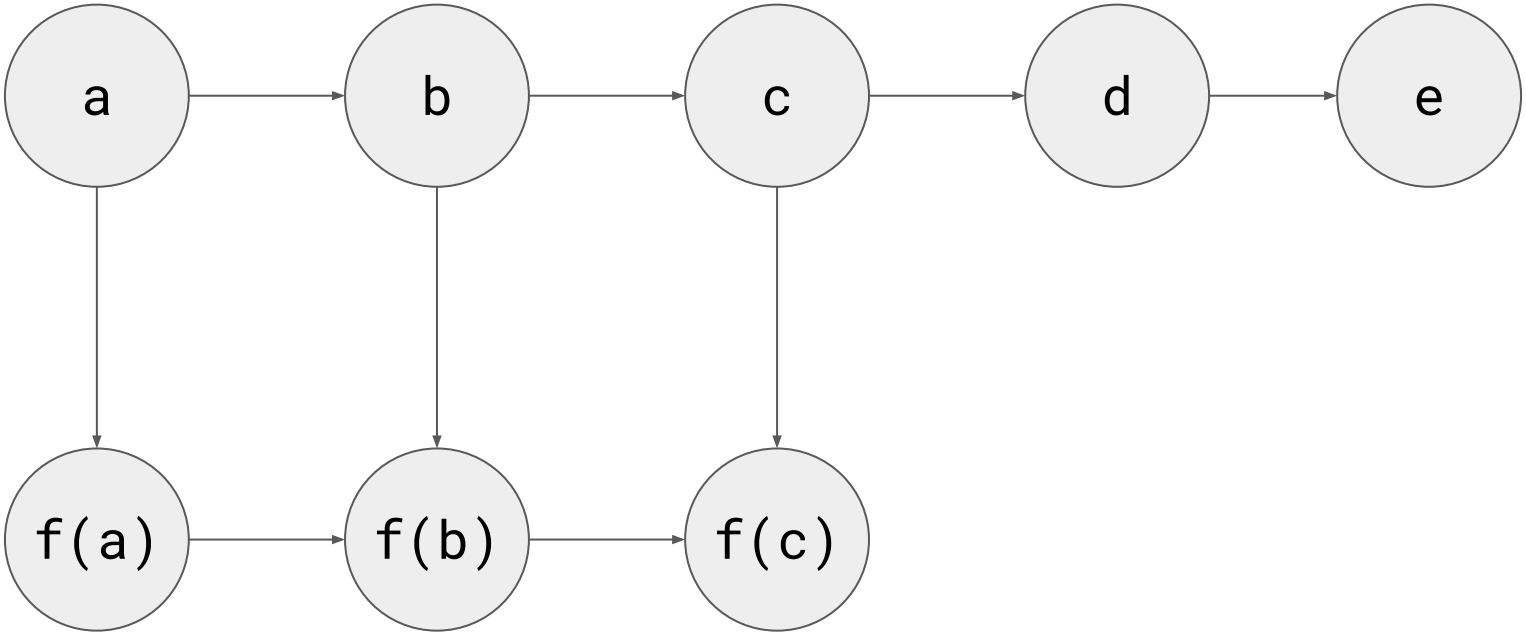
map f [a, b, c, d, e]



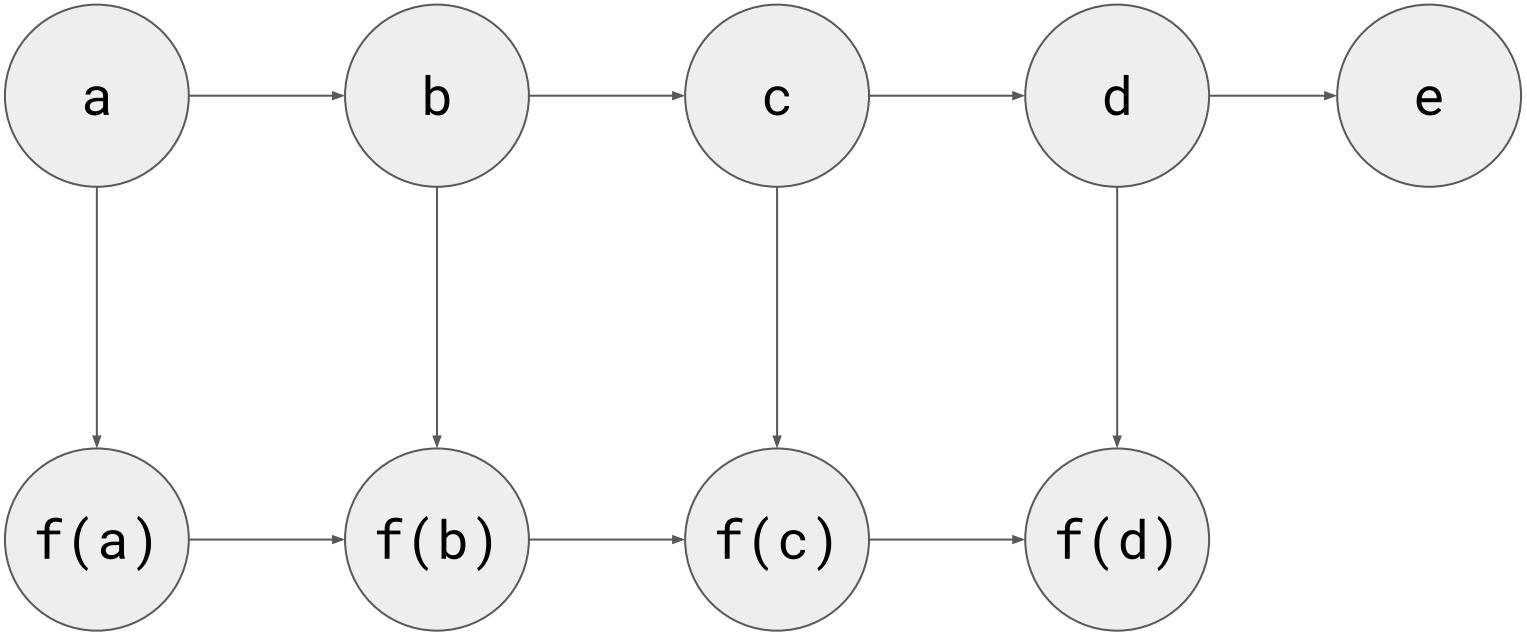
map f [a, b, c, d, e]



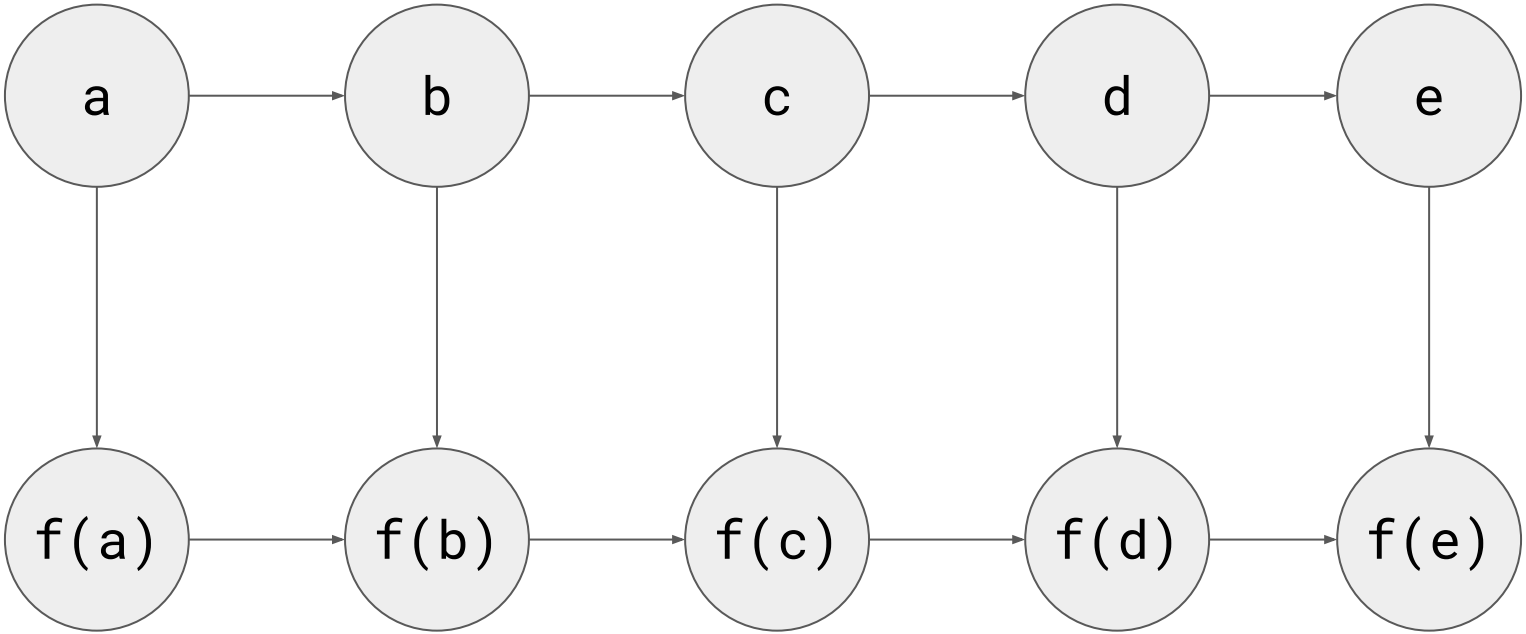
map f [a, b, c, d, e]



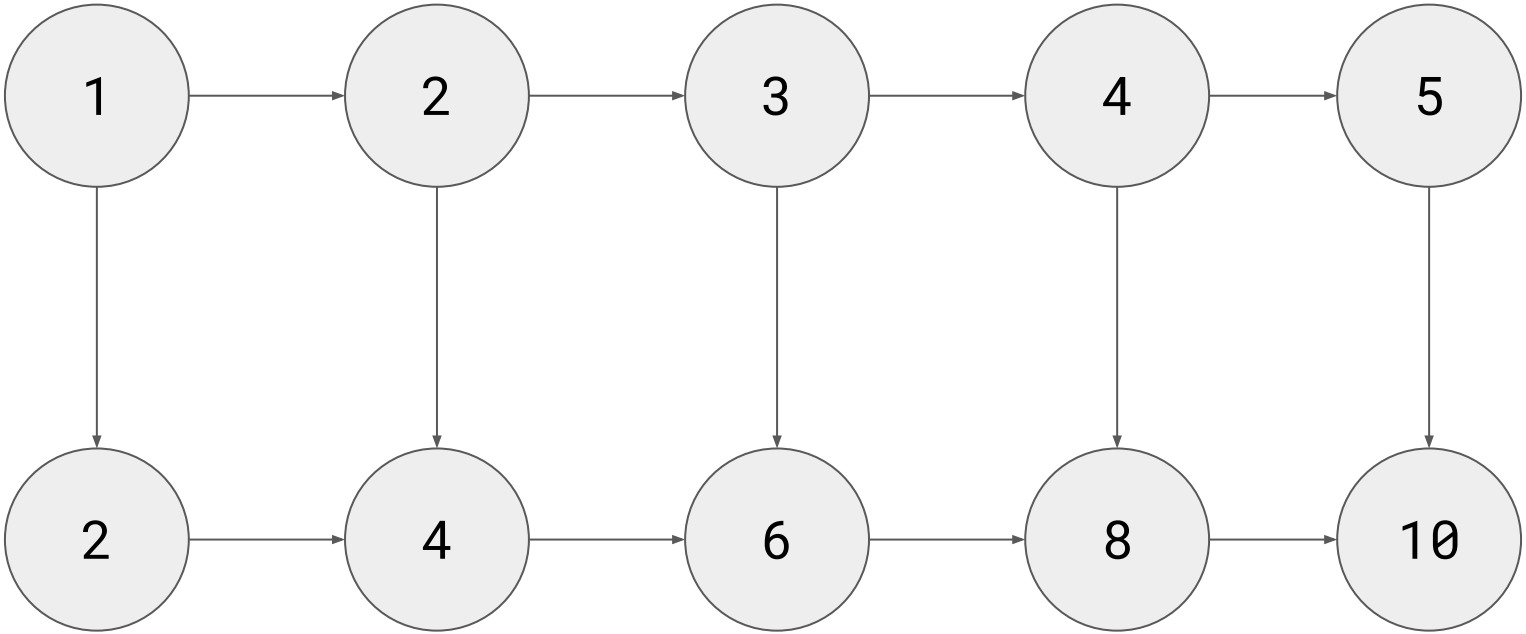
map f [a, b, c, d, e]



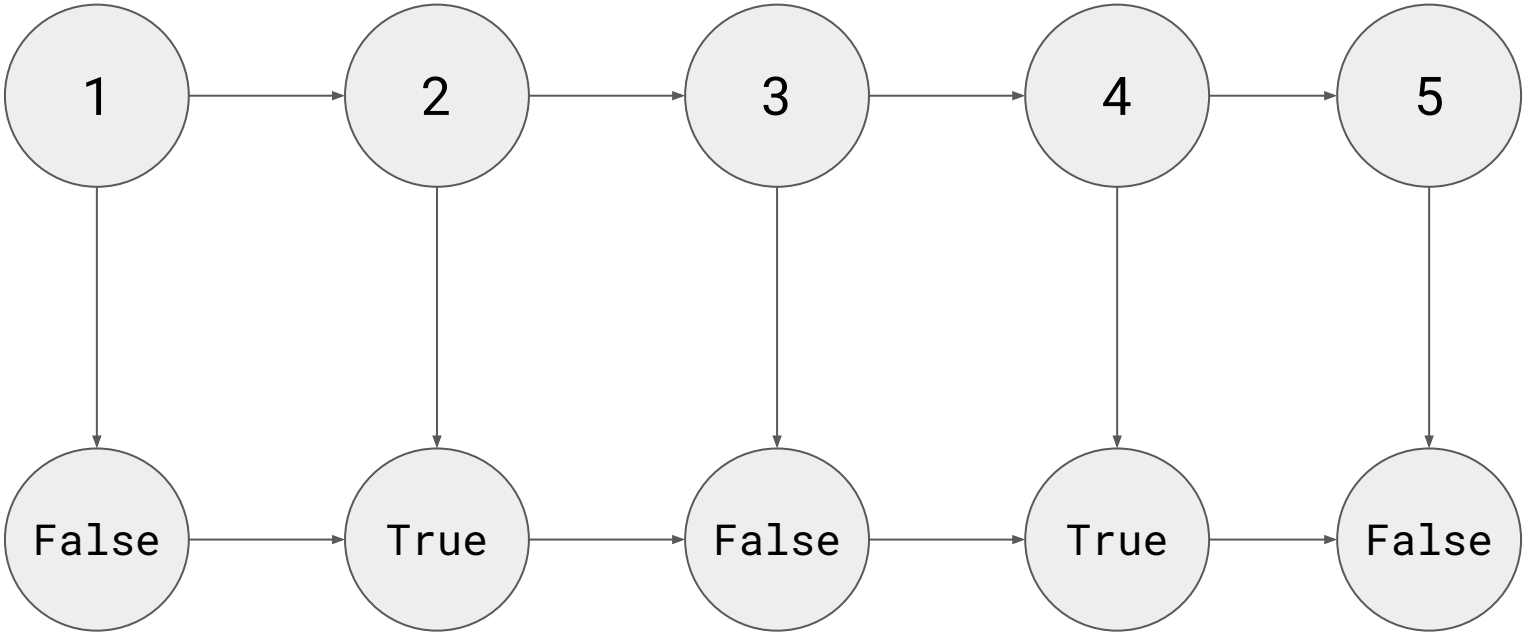
map f [a, b, c, d, e]



map (*2) [1, 2, 3, 4, 5]



map even [1, 2, 3, 4, 5]



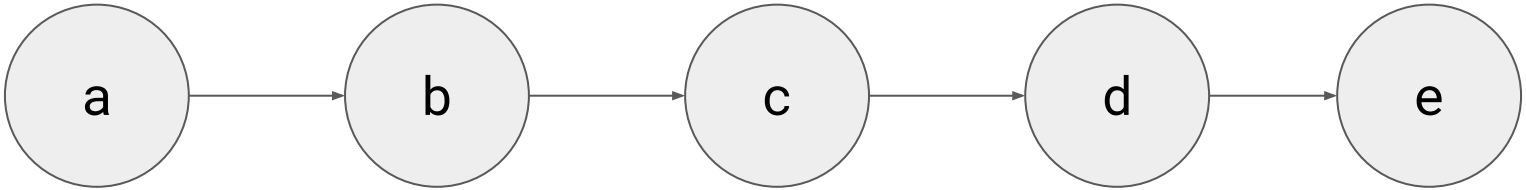
filter

filter f list

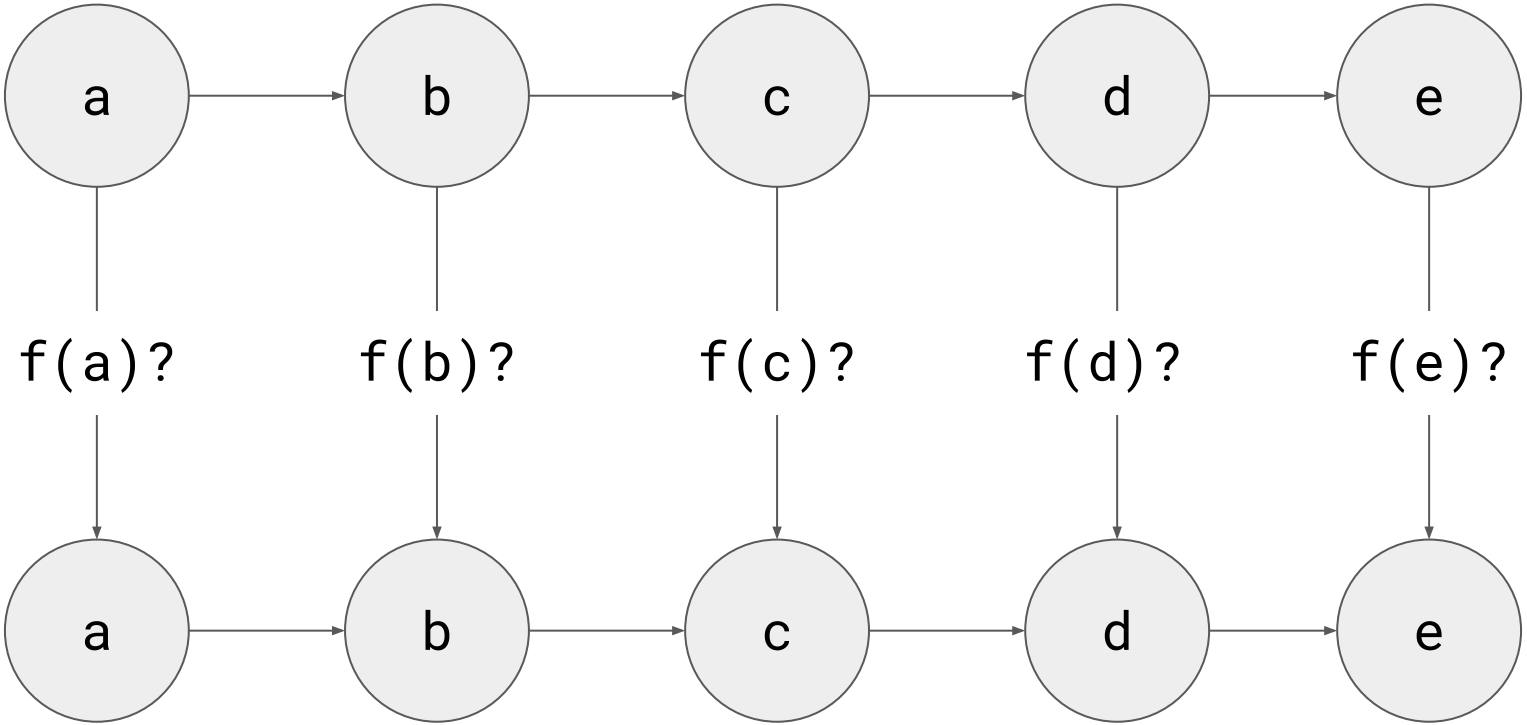
```
filter :: (a -> Bool) -> [a] -> [a]

filter _ [] = []
filter p (x:xs)
  | p x = x : filter p xs
  | otherwise = filter p xs
```

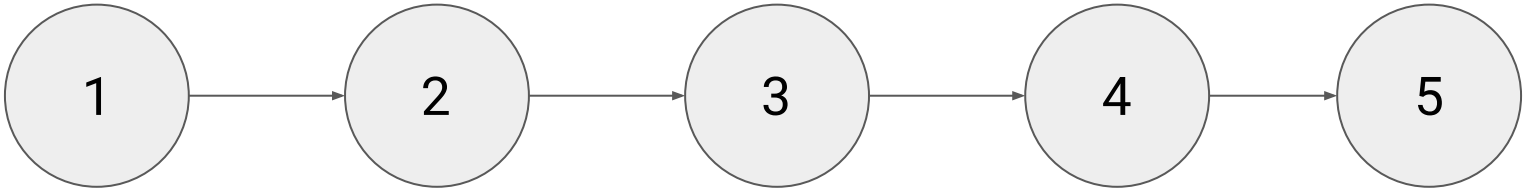
[a, b, c, d, e]



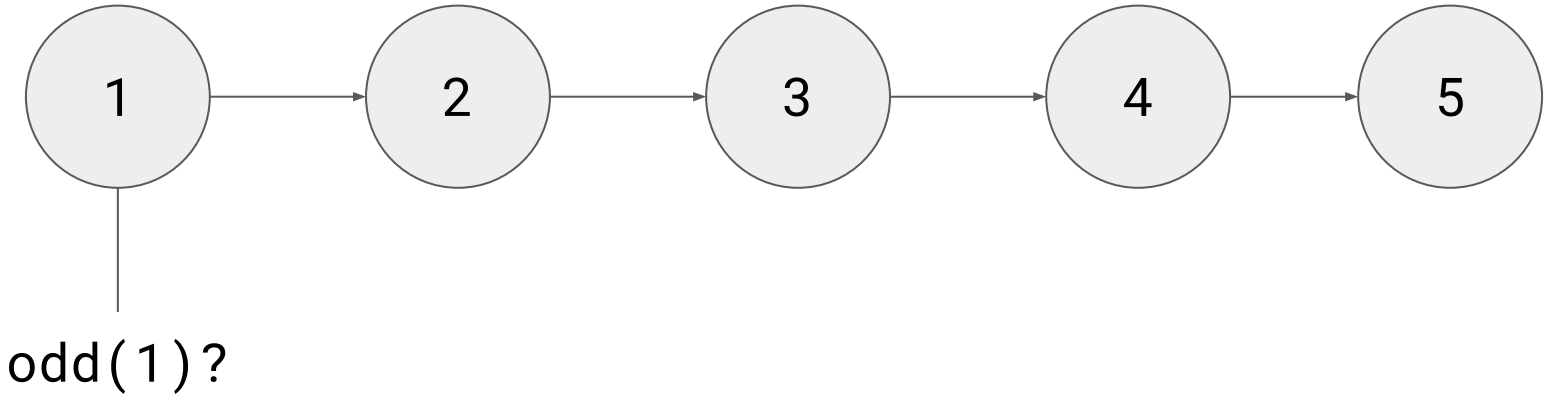
filter f [a, b, c, d, e]



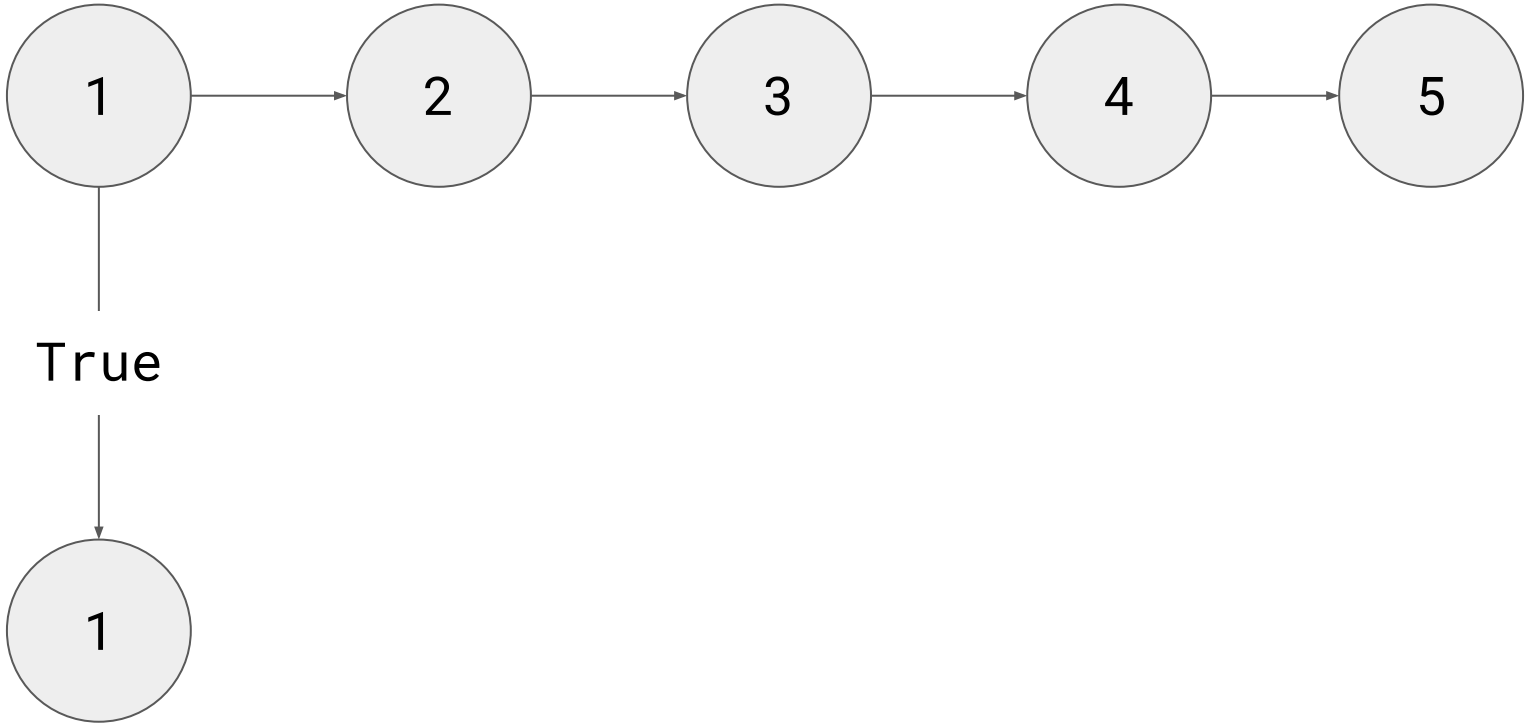
filter odd [1, 2, 3, 4, 5]



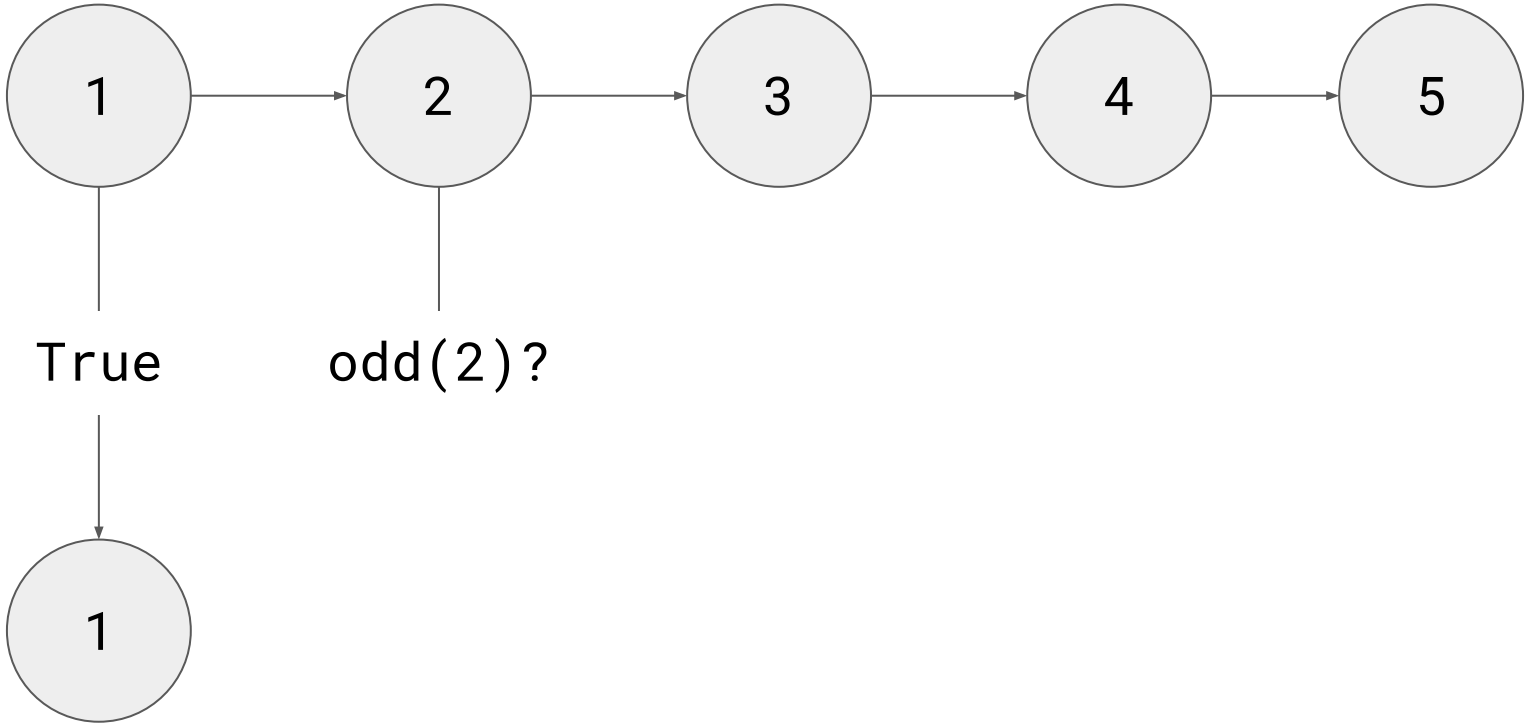
filter odd [1, 2, 3, 4, 5]



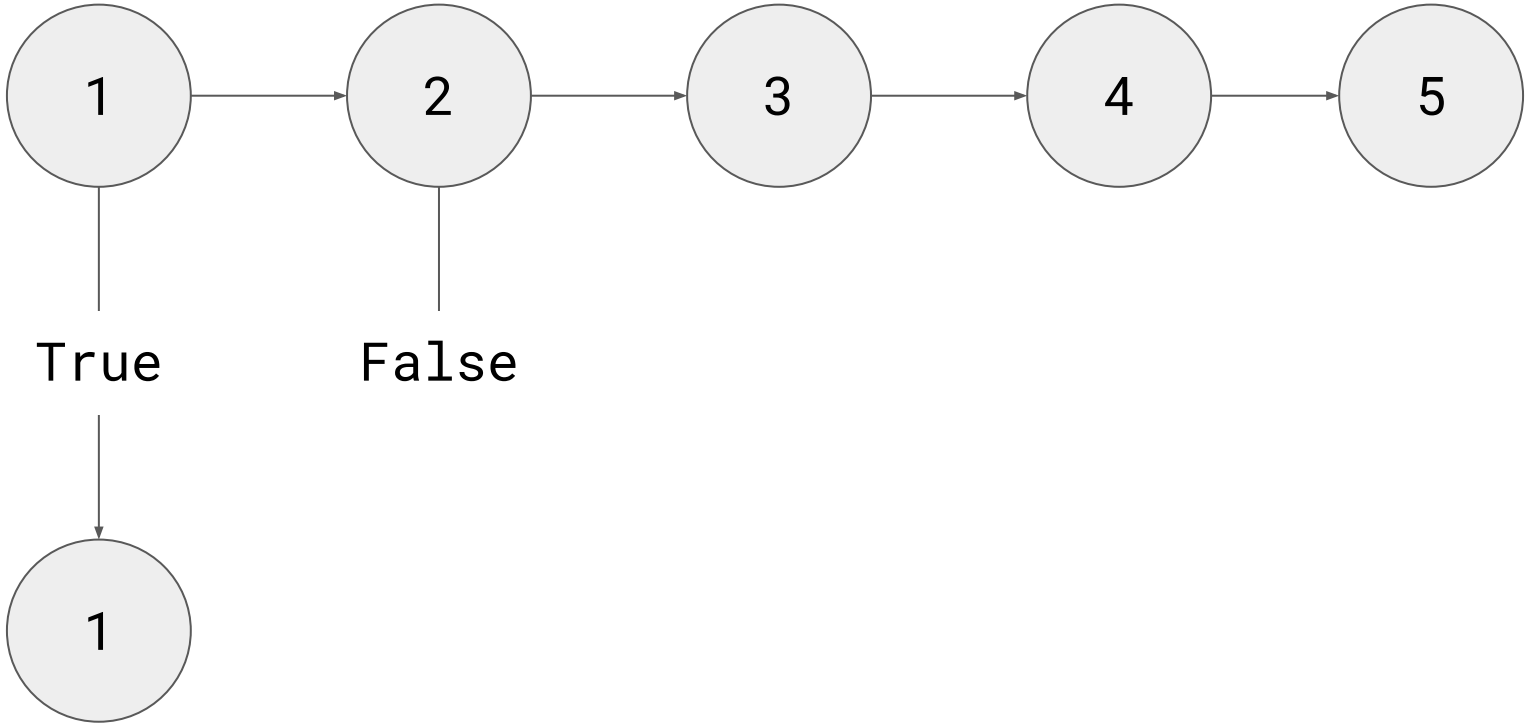
filter odd [1, 2, 3, 4, 5]



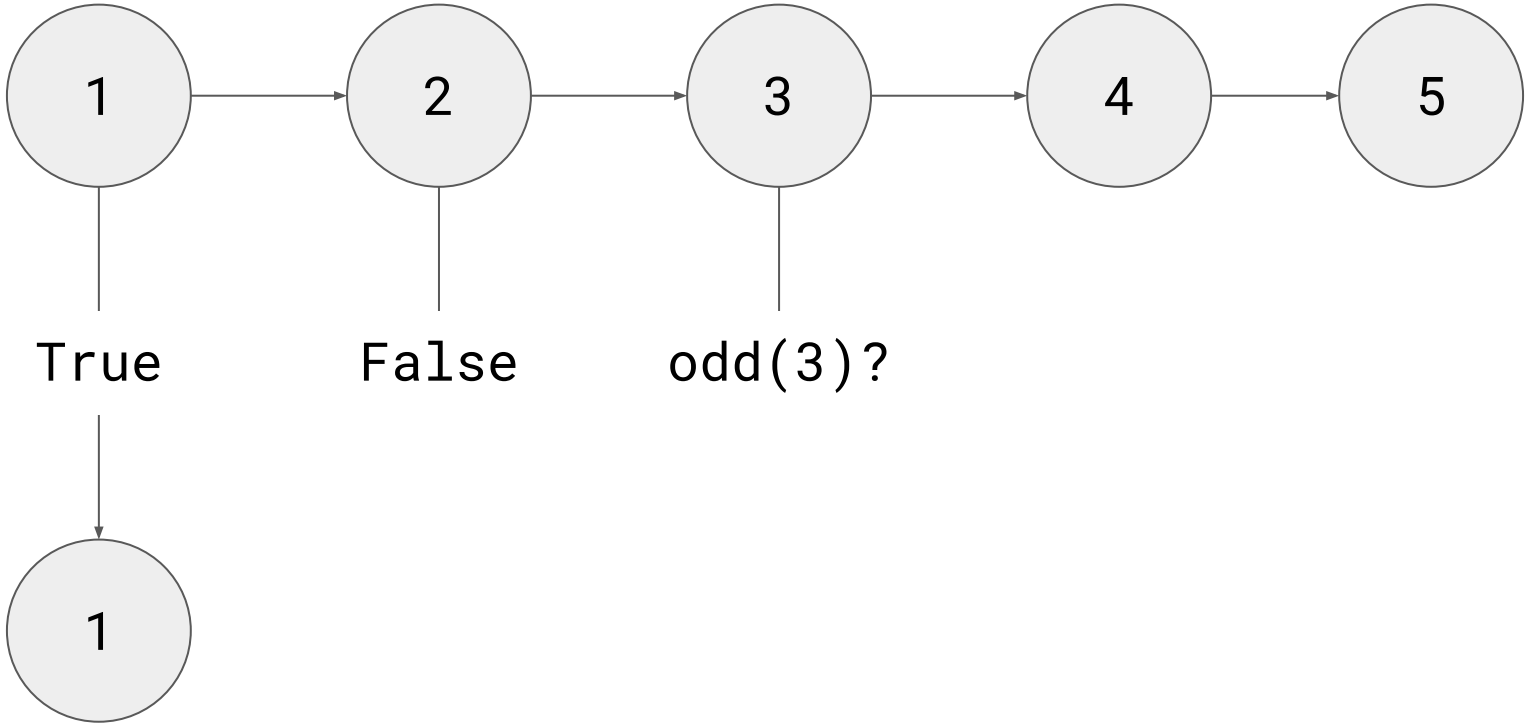
filter odd [1, 2, 3, 4, 5]



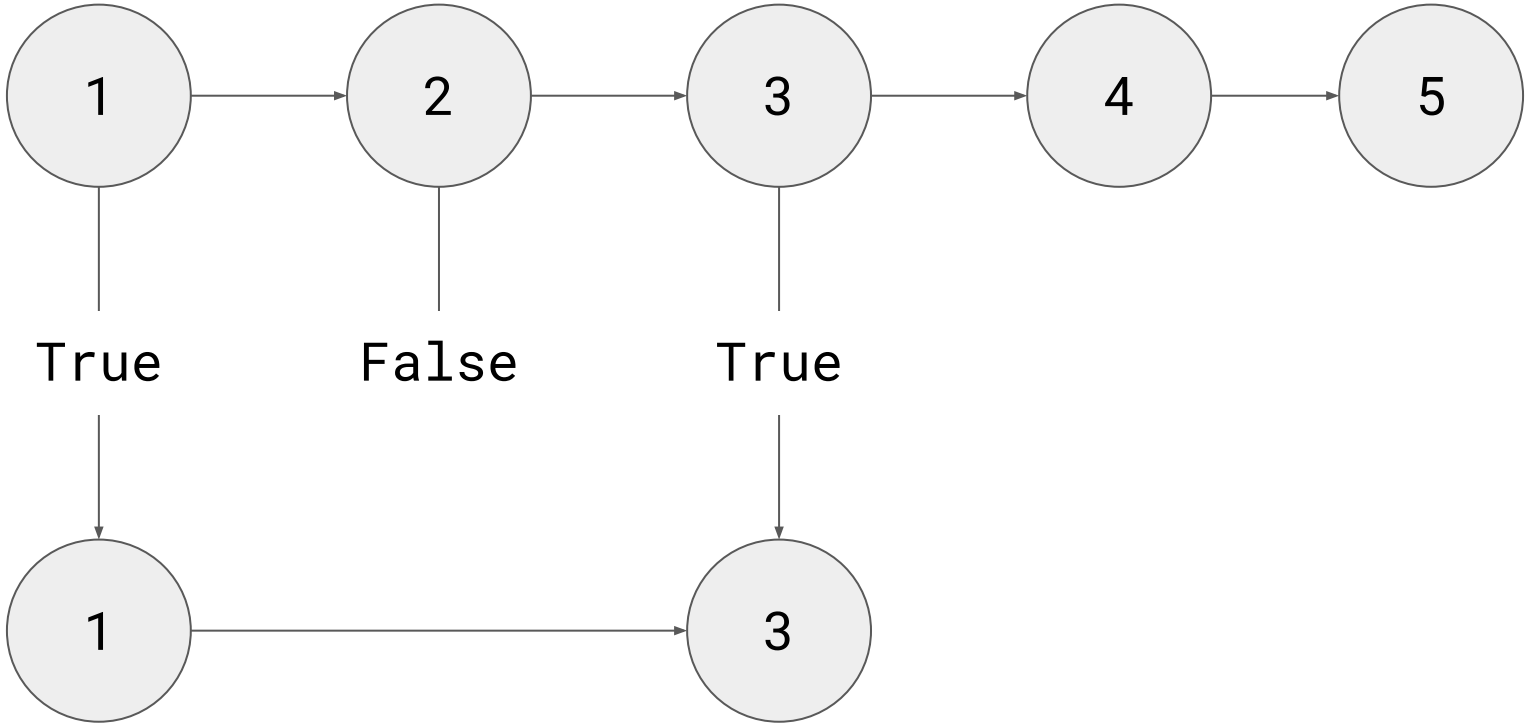
filter odd [1, 2, 3, 4, 5]



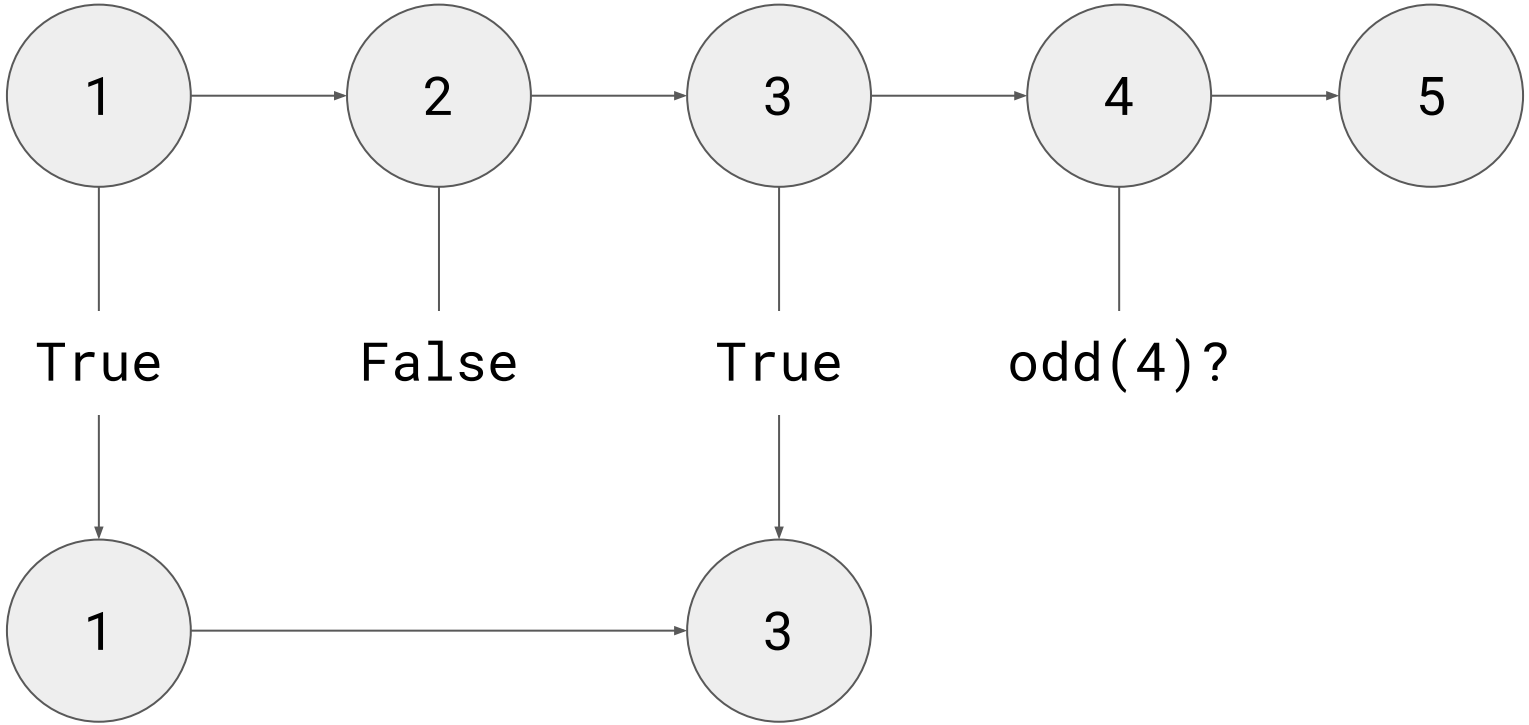
filter odd [1, 2, 3, 4, 5]



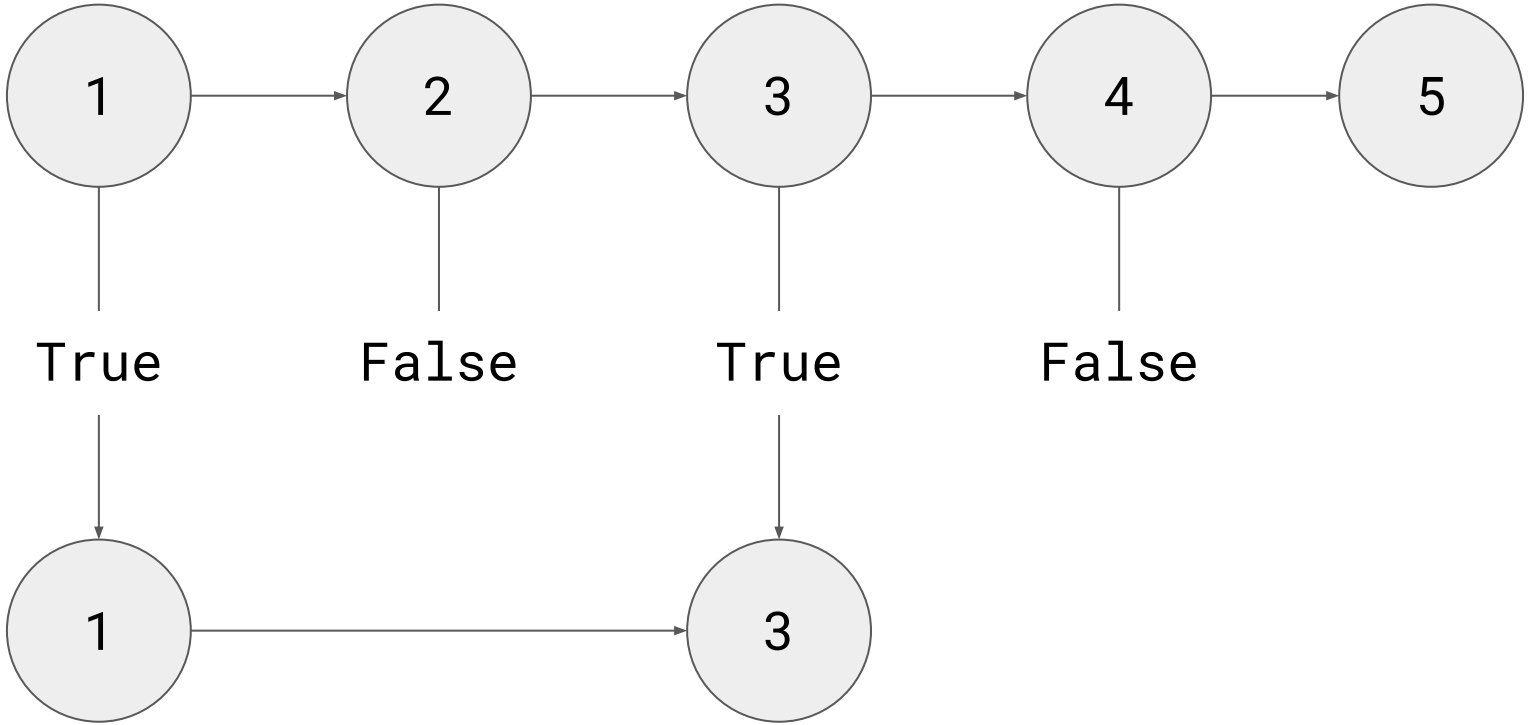
filter odd [1, 2, 3, 4, 5]



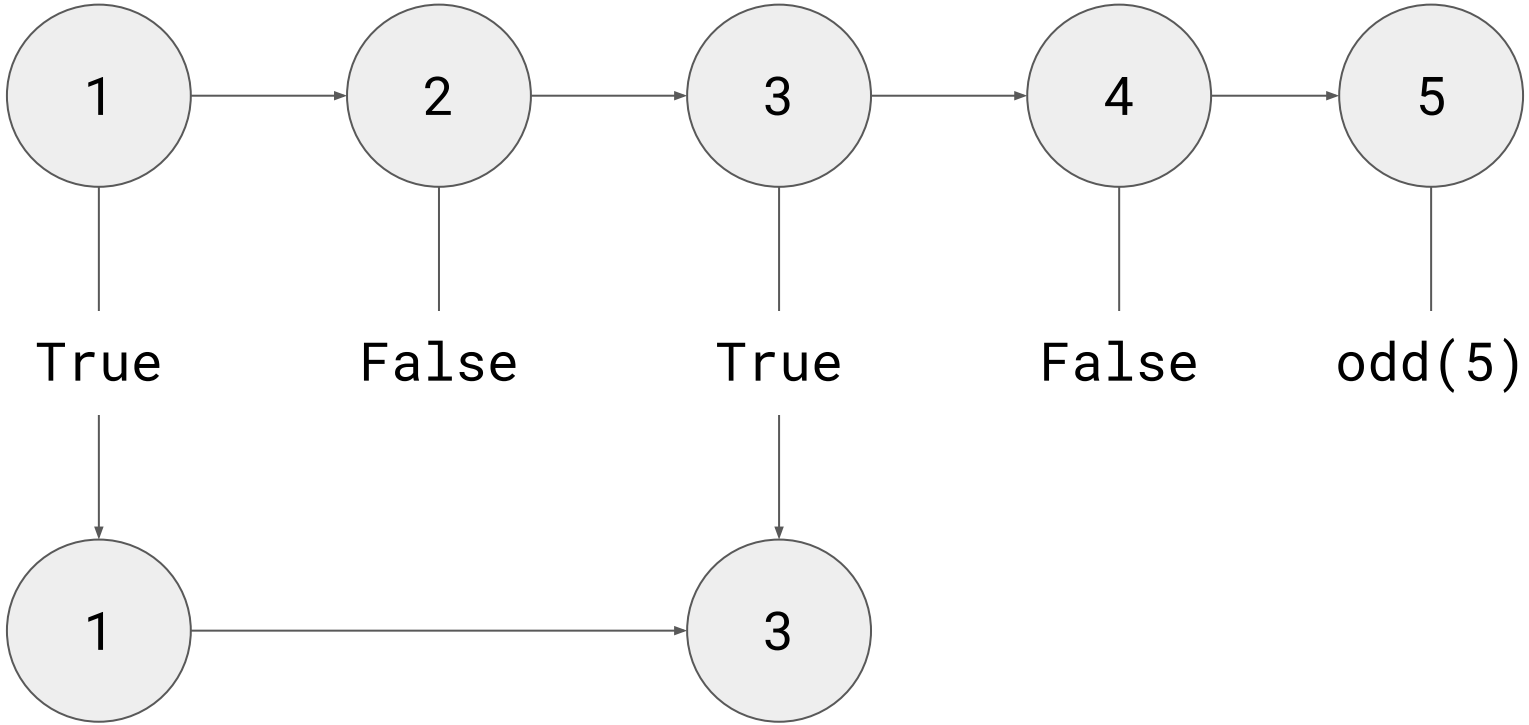
filter odd [1, 2, 3, 4, 5]



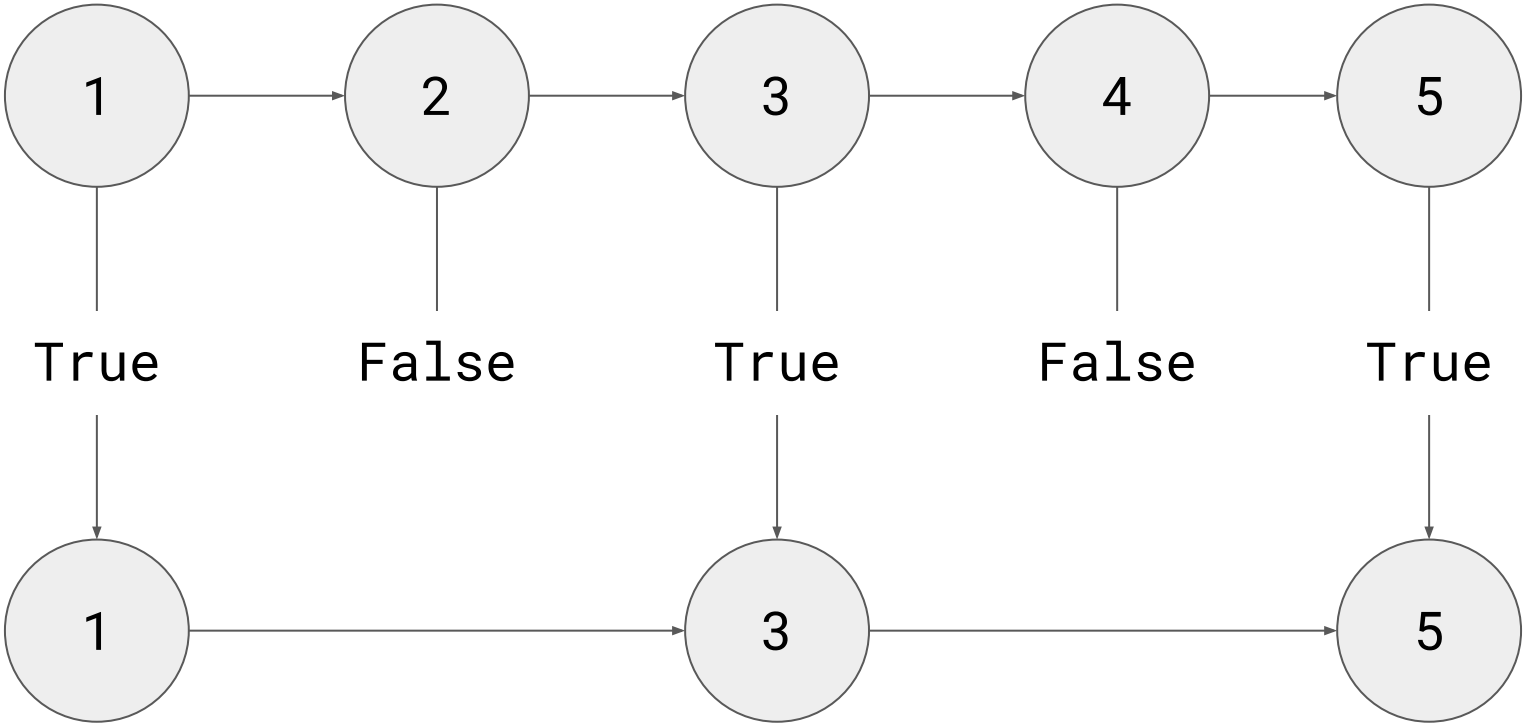
filter odd [1, 2, 3, 4, 5]



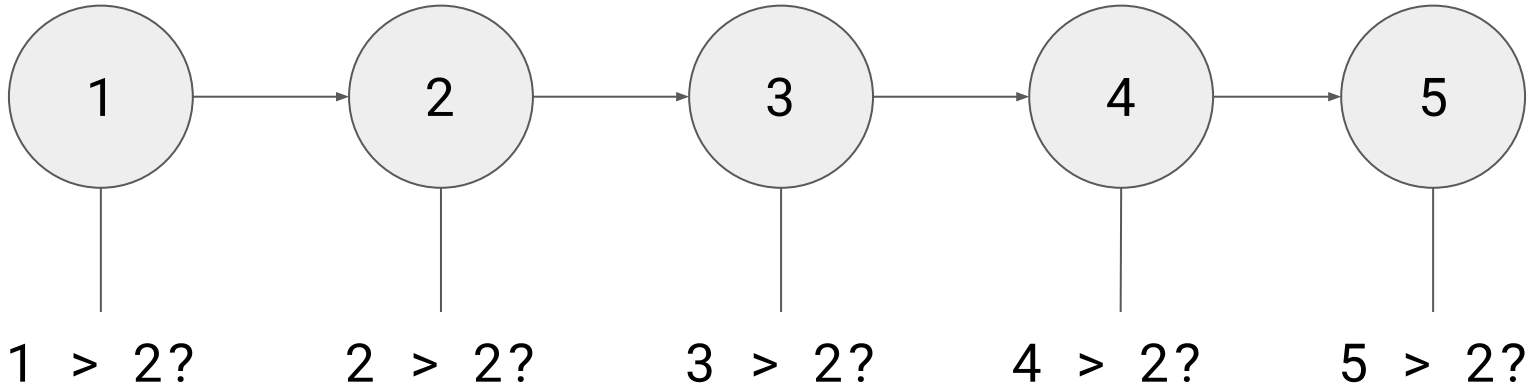
filter odd [1, 2, 3, 4, 5]



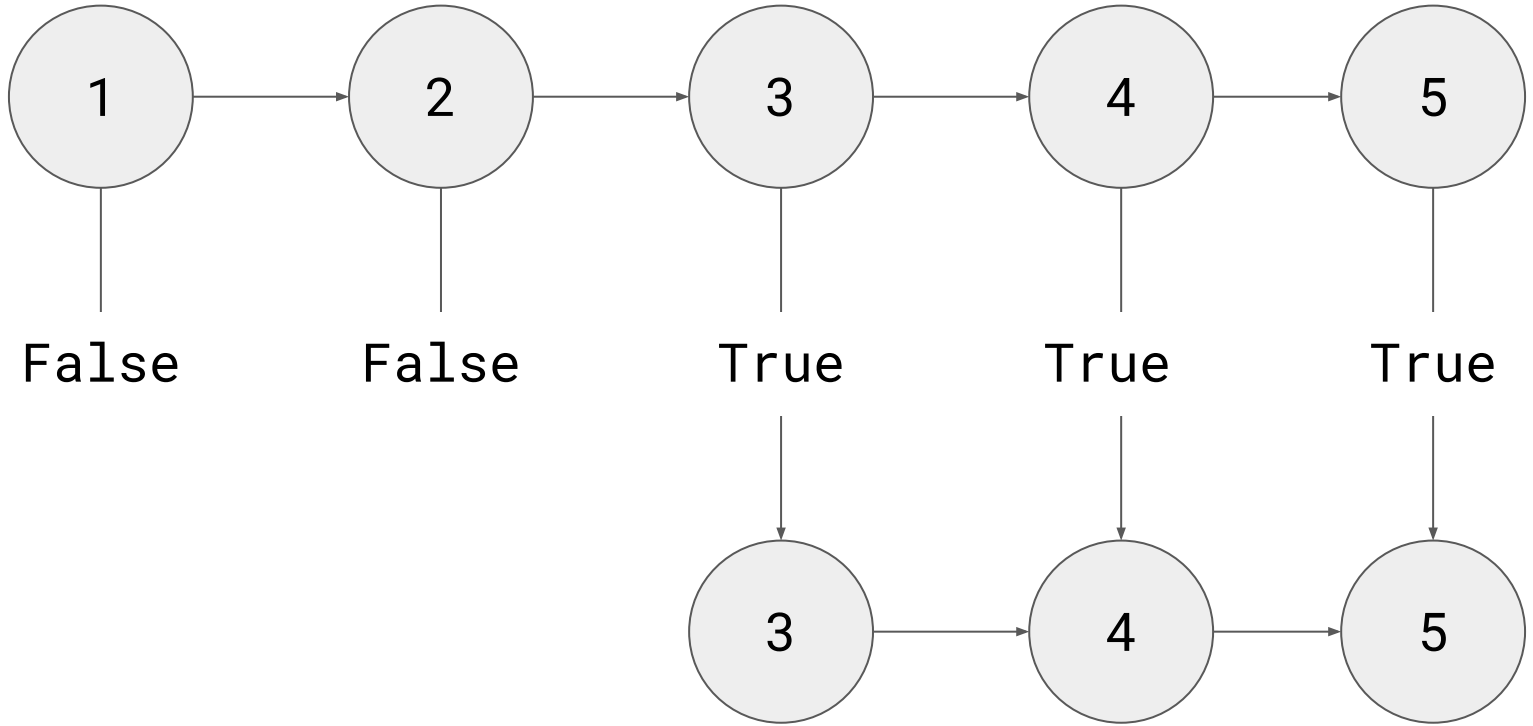
filter odd [1, 2, 3, 4, 5]



filter (>2) [a, b, c, d, e]



filter (>2) [a, b, c, d, e]

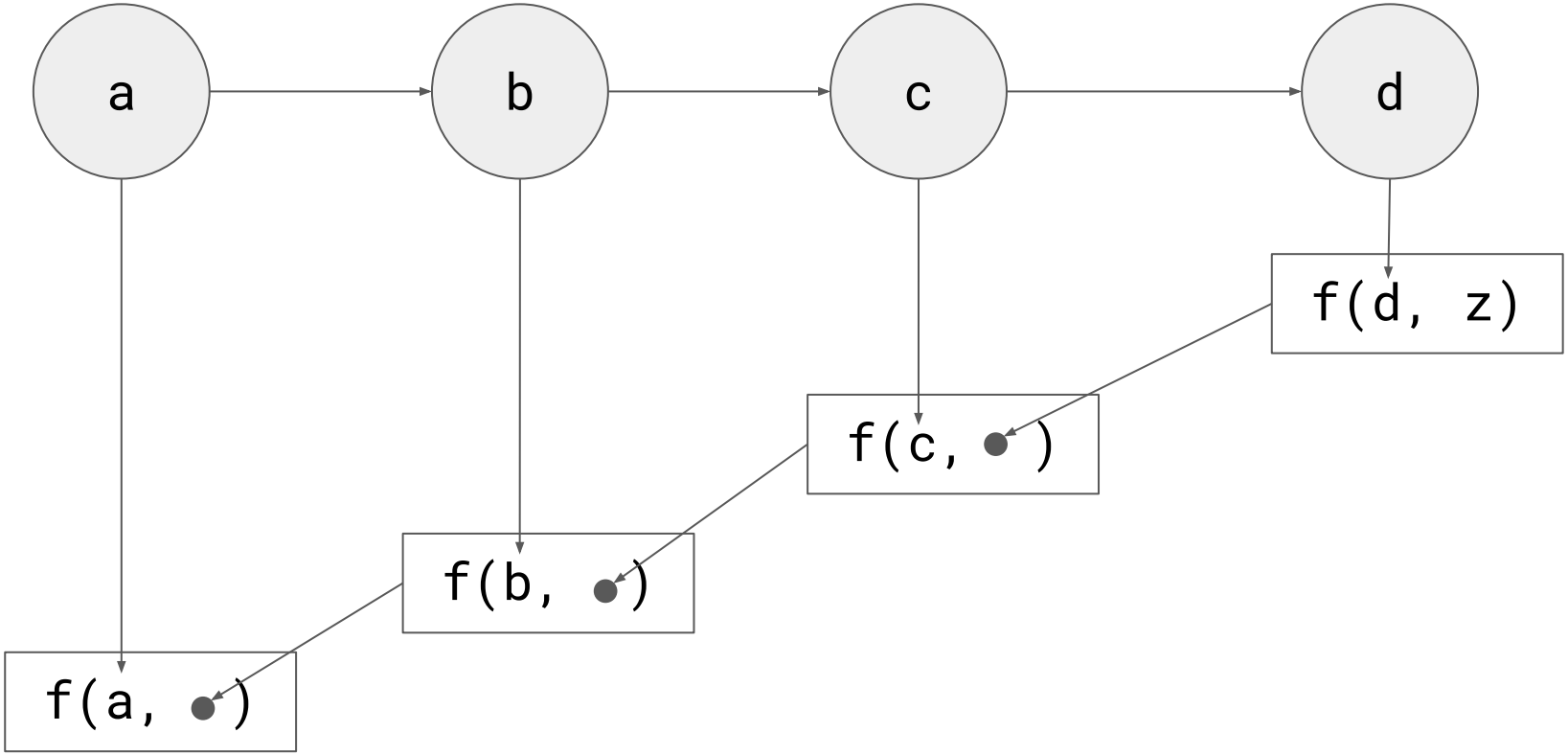


Are those enough?

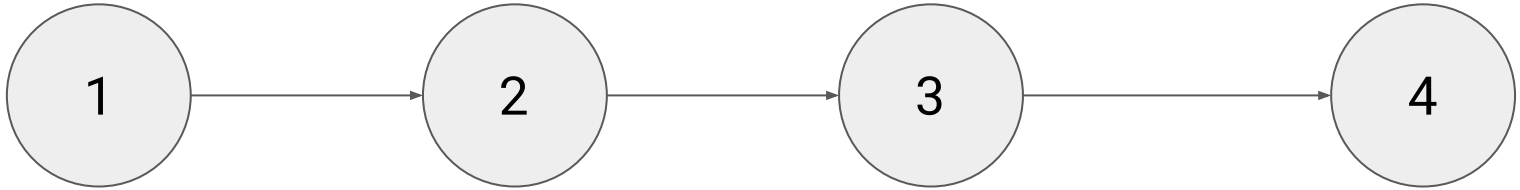
foldr

foldr f z list

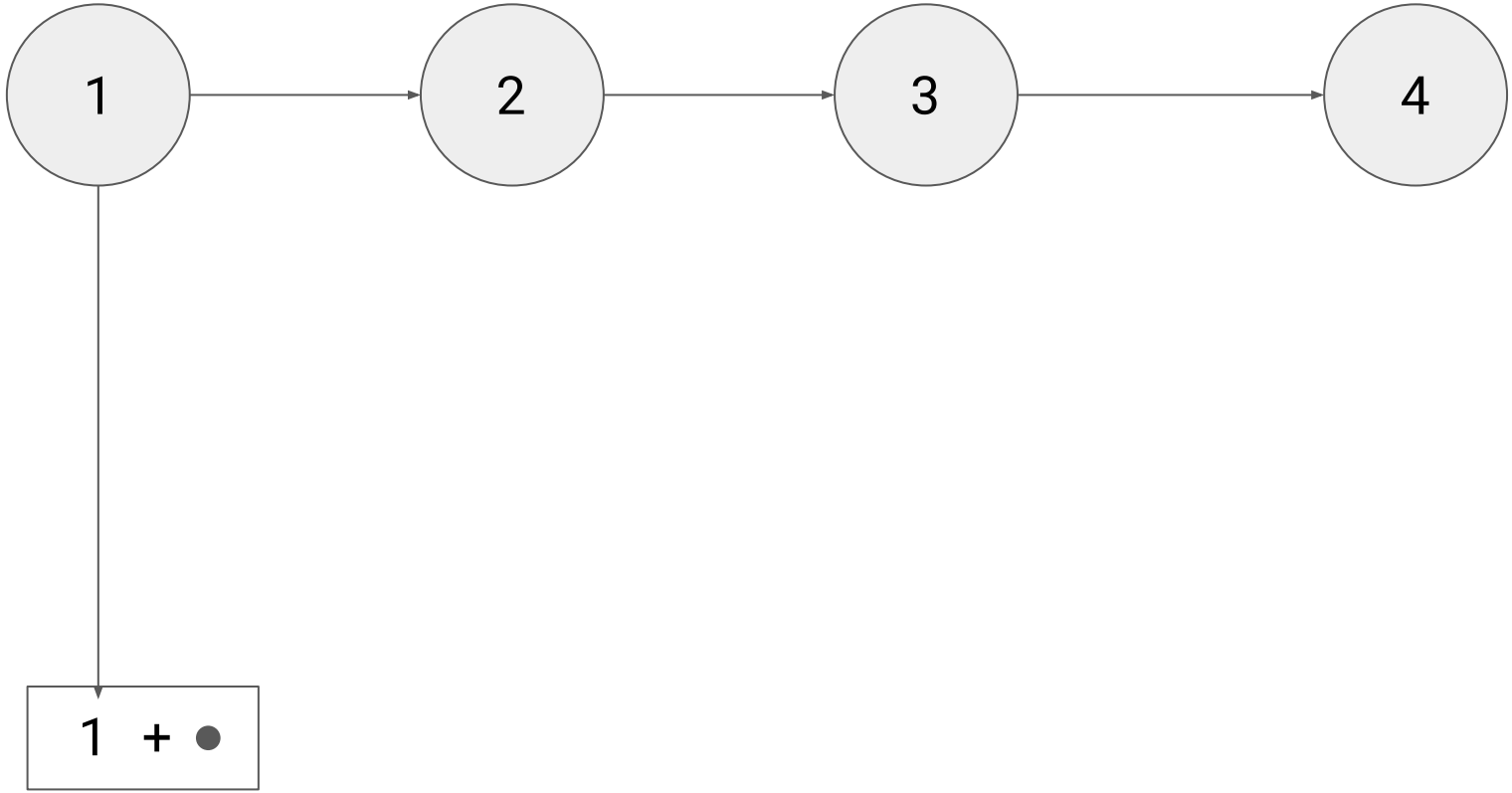
```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f z []     = z
foldr f z (x:xs) = x `f` (foldr f z xs)
```



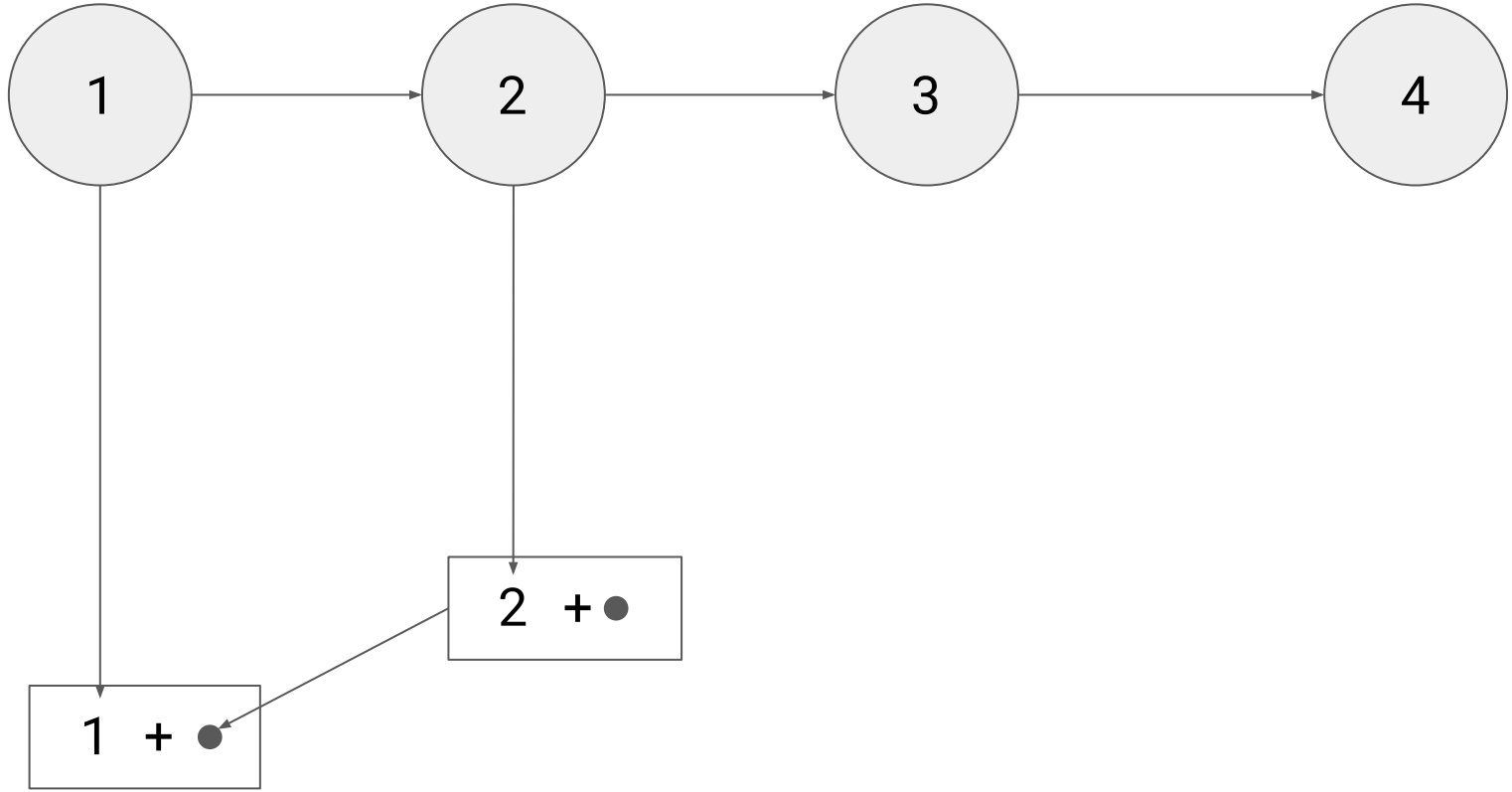

```
foldr f z [a, b, c, d]
```



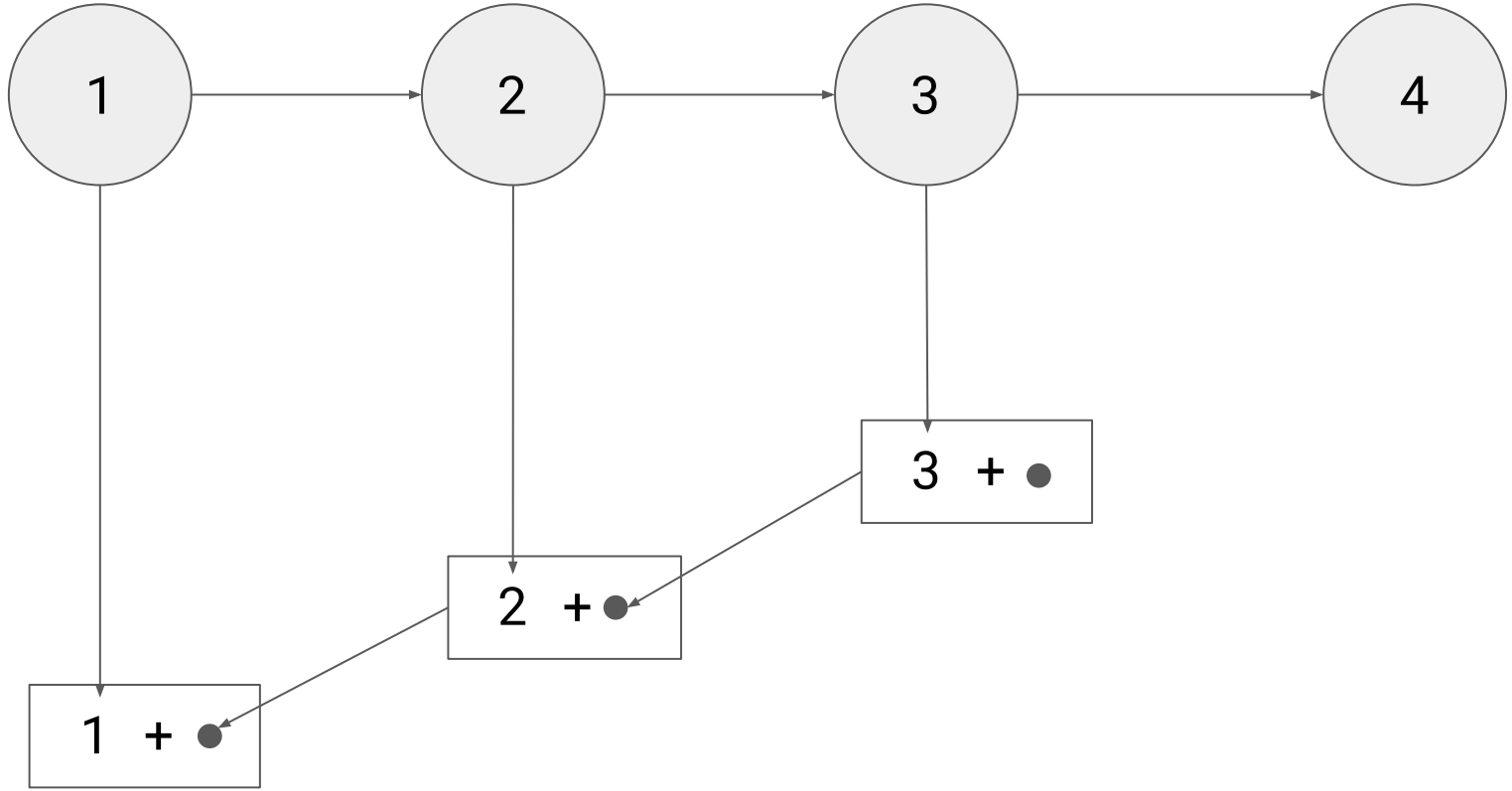

```
foldr (+) 0 [1, 2, 3, 4]
```



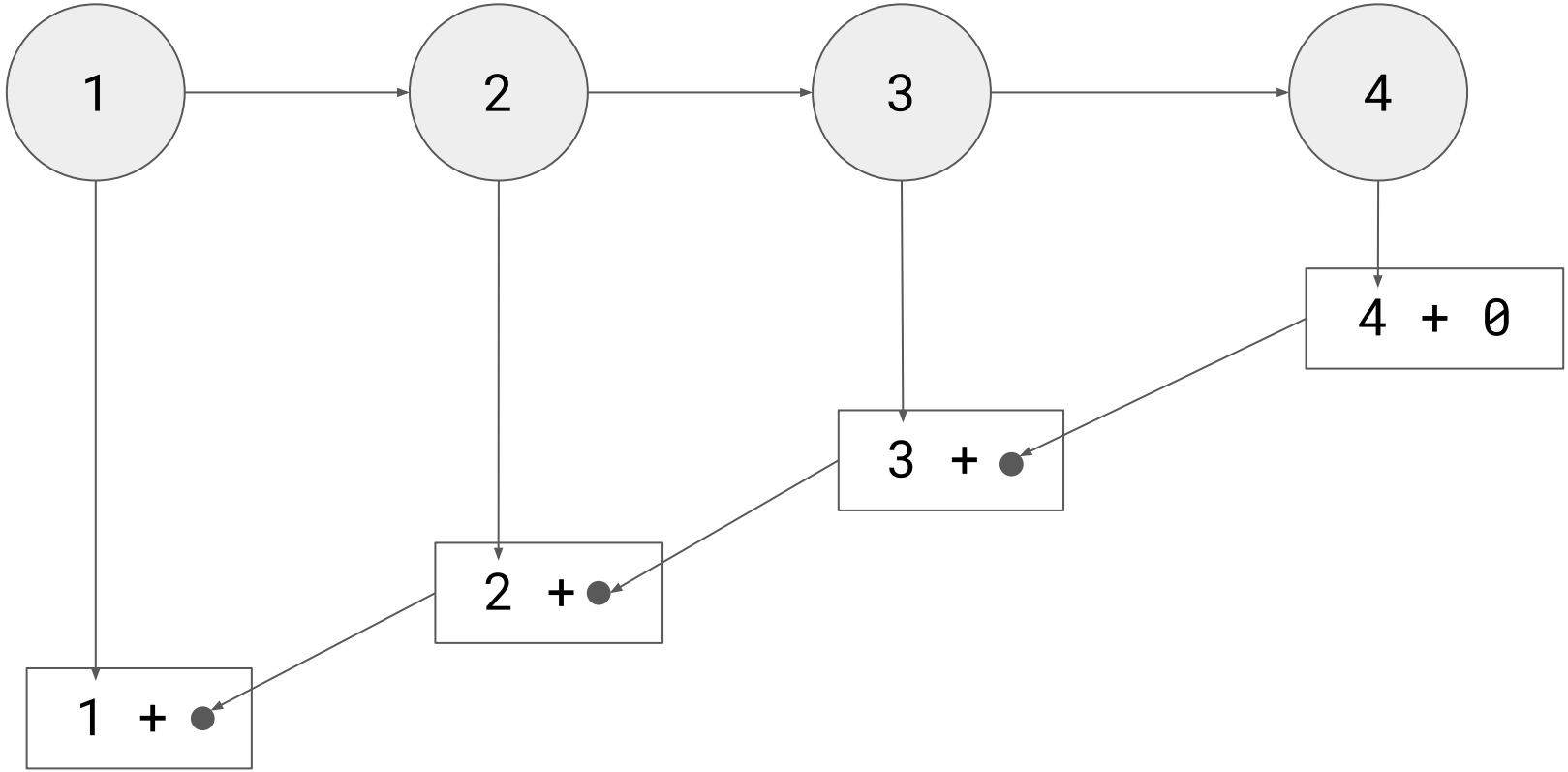

```
foldr (+) 0 [1, 2, 3, 4]
```



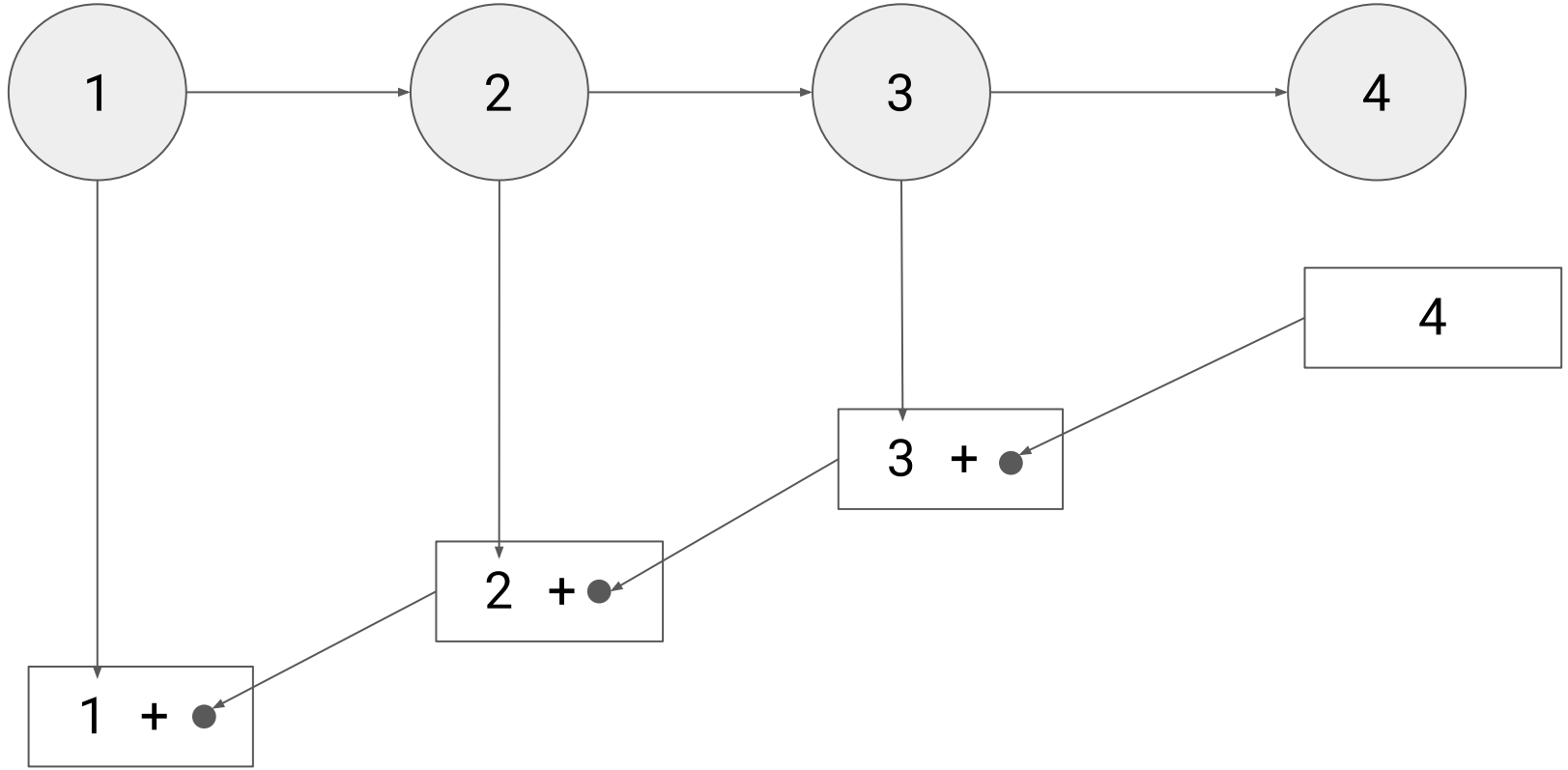

```
foldr (+) 0 [1, 2, 3, 4]
```



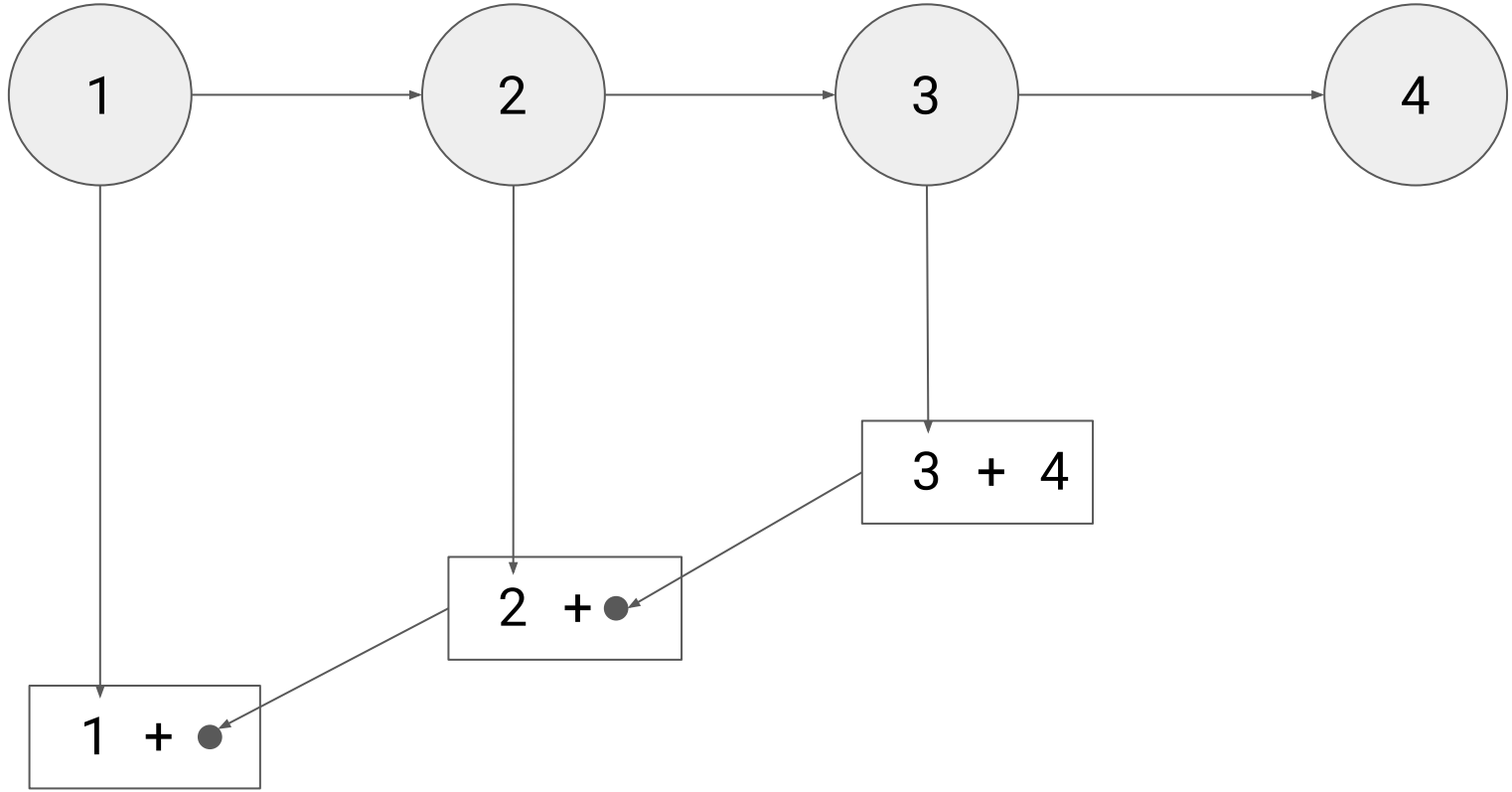

```
foldr (+) 0 [1, 2, 3, 4]
```



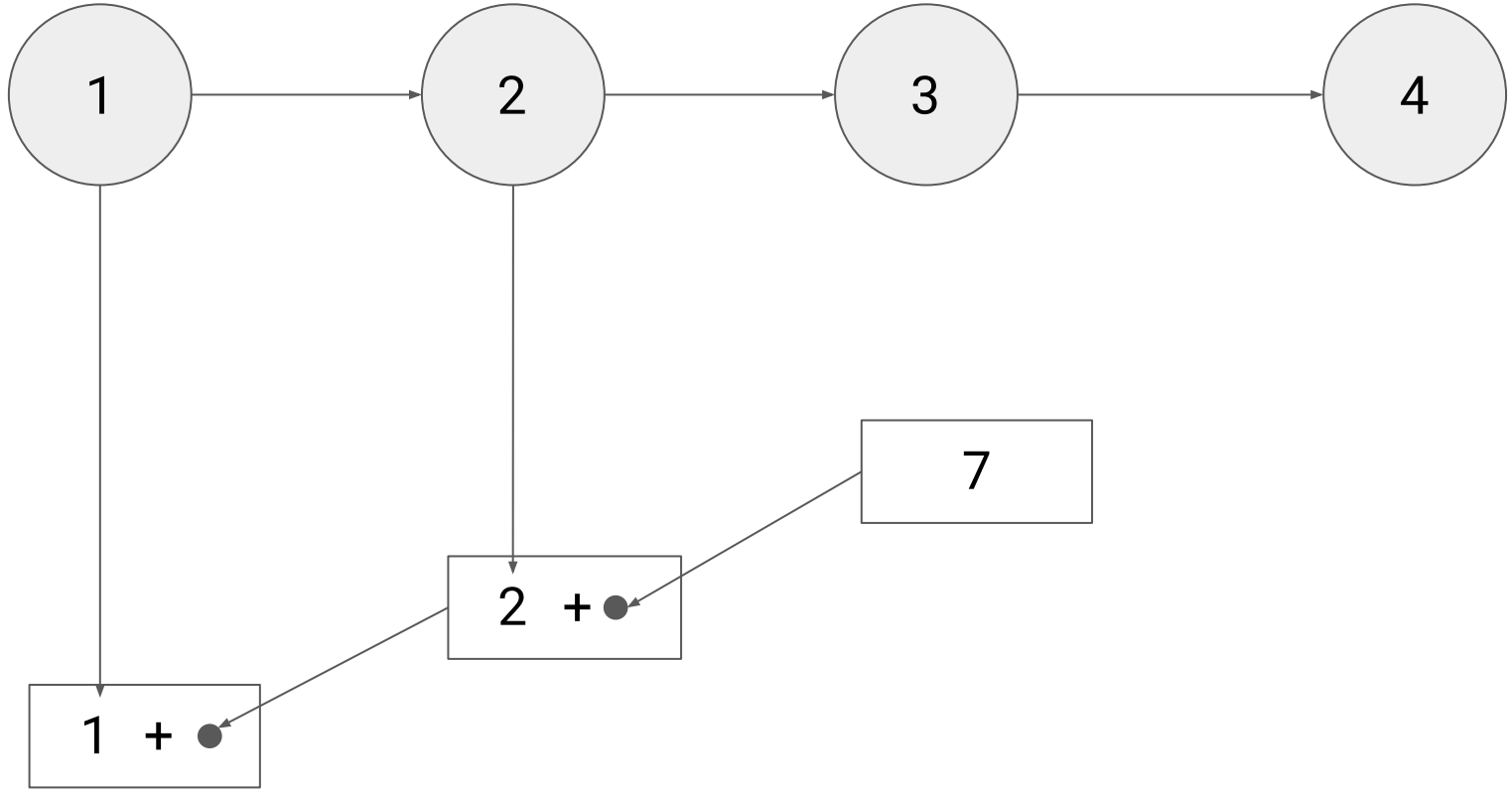

```
foldr (+) 0 [1, 2, 3, 4]
```



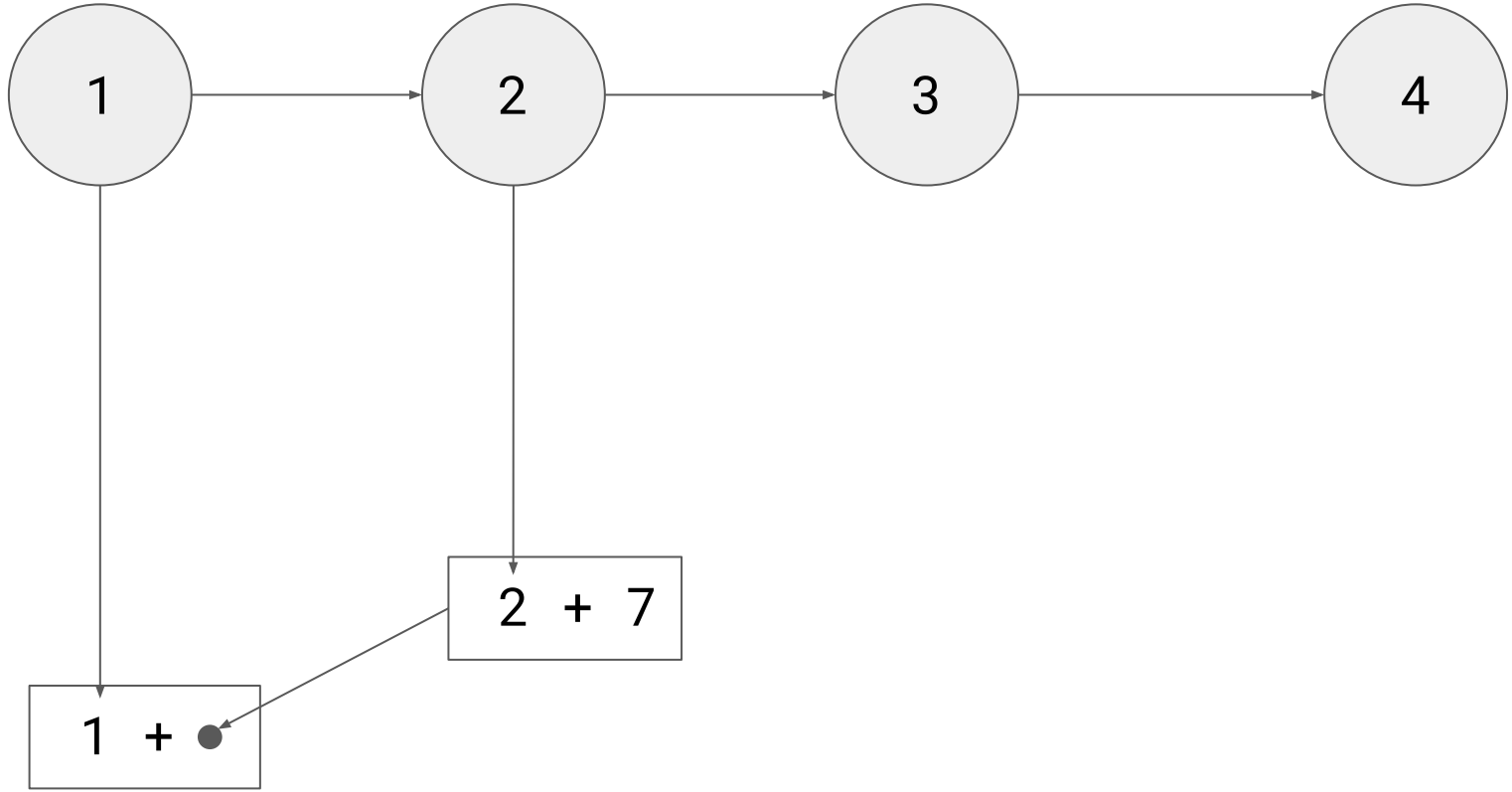

```
foldr (+) 0 [1, 2, 3, 4]
```



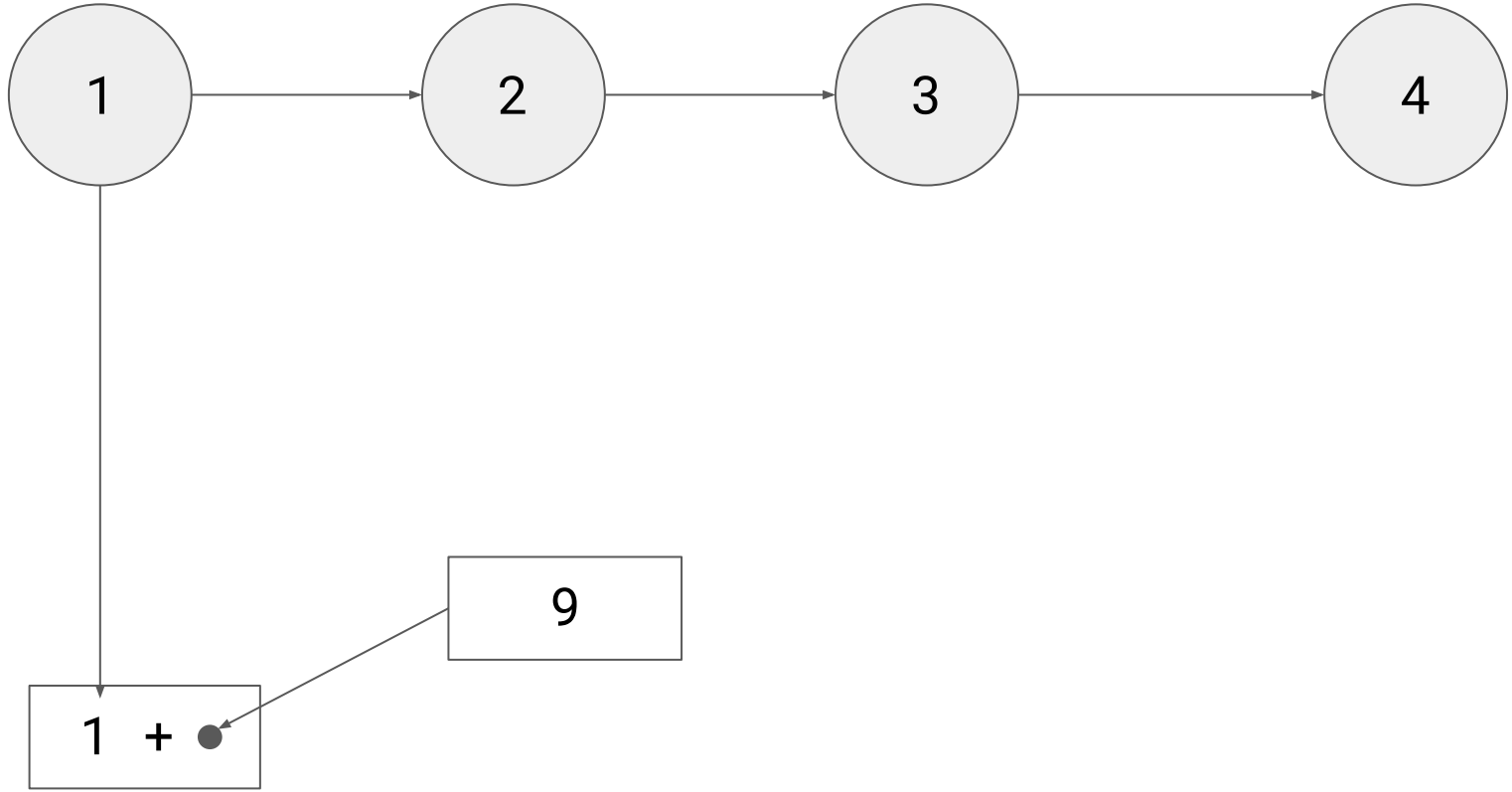

```
foldr (+) 0 [1, 2, 3, 4]
```



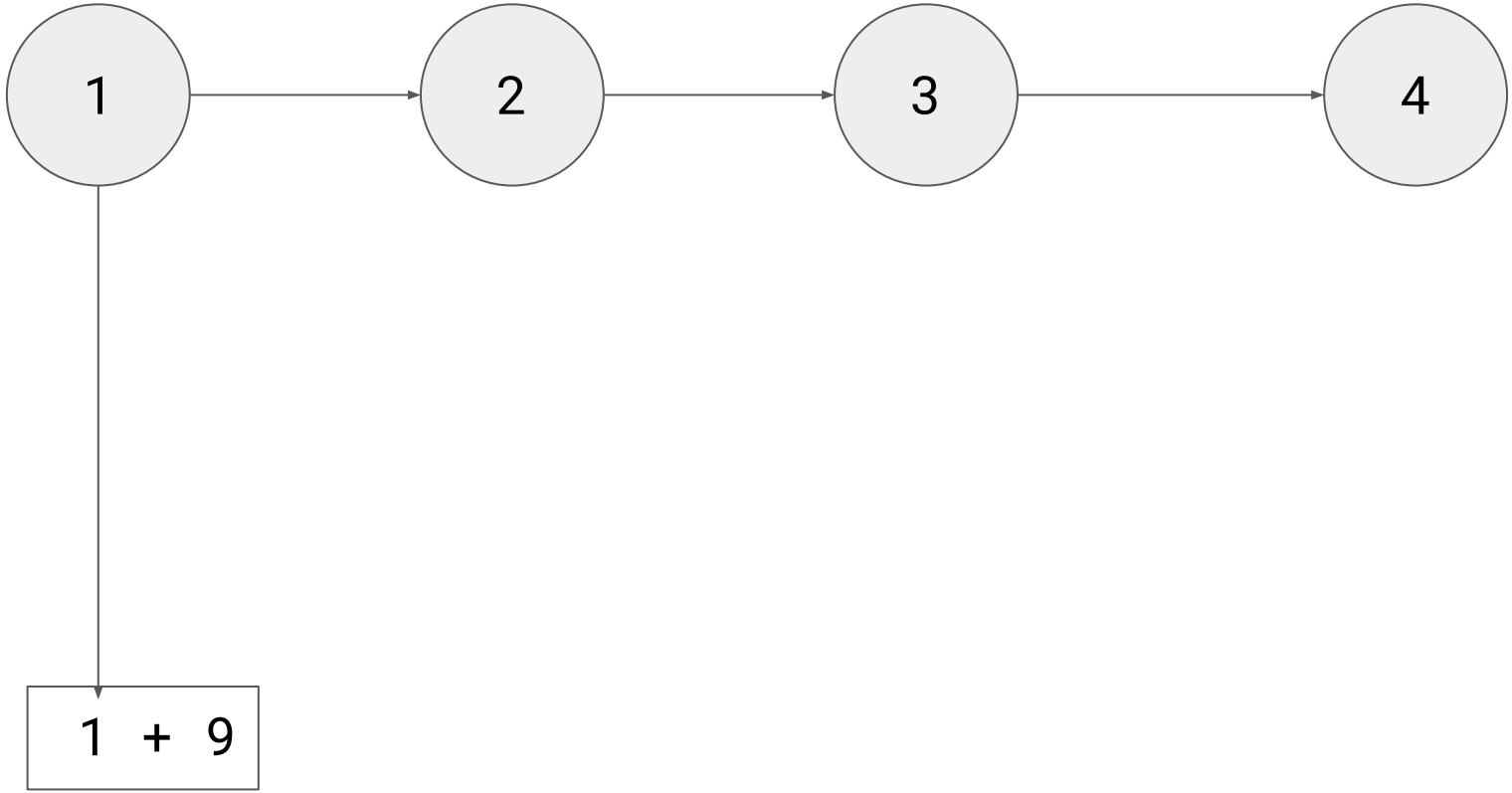

```
foldr (+) 0 [1, 2, 3, 4]
```



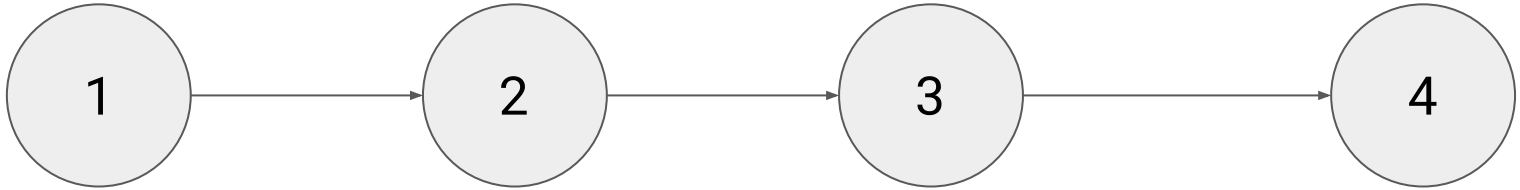

```
foldr (+) 0 [1, 2, 3, 4]
```




```
foldr (+) 0 [1, 2, 3, 4]
```

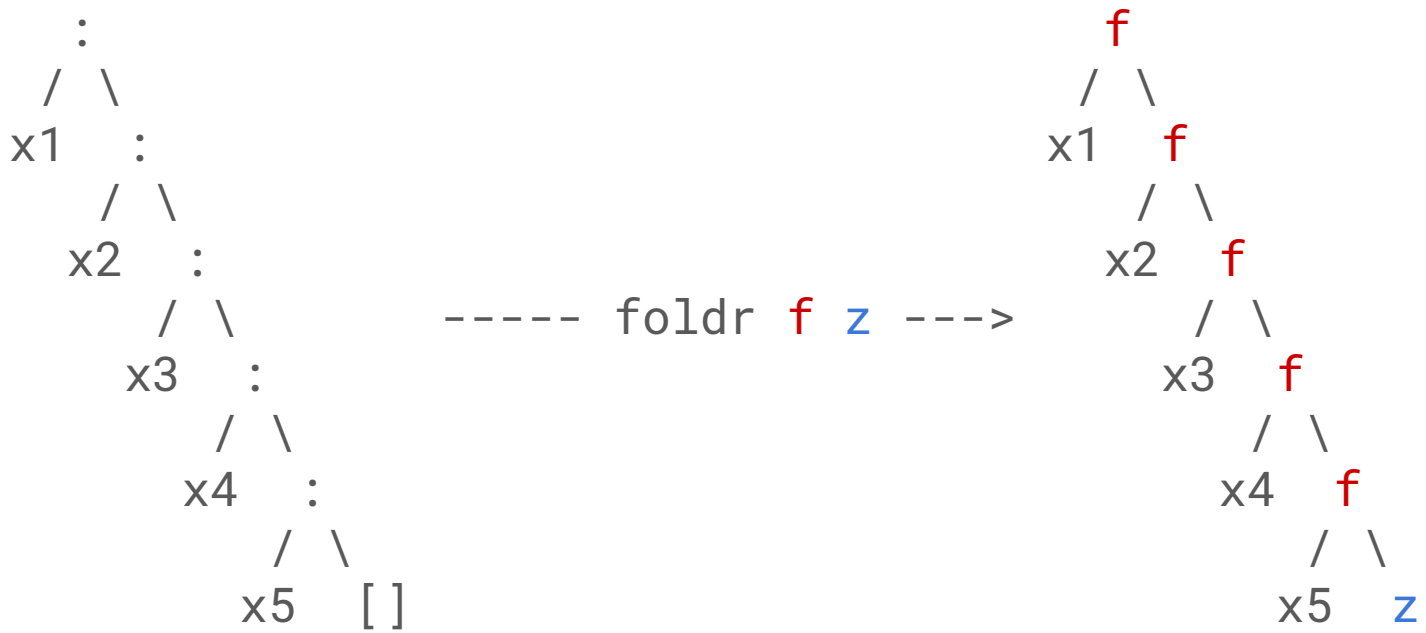


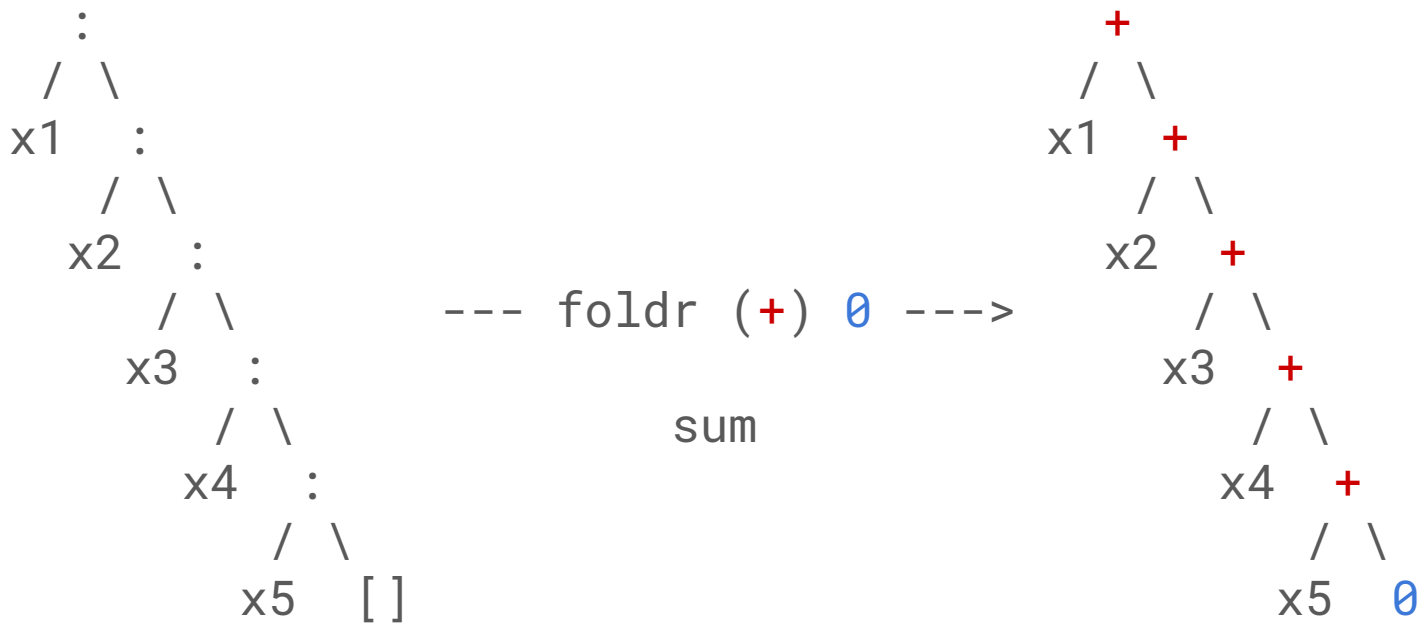

```
foldr (+) 0 [1, 2, 3, 4]
```

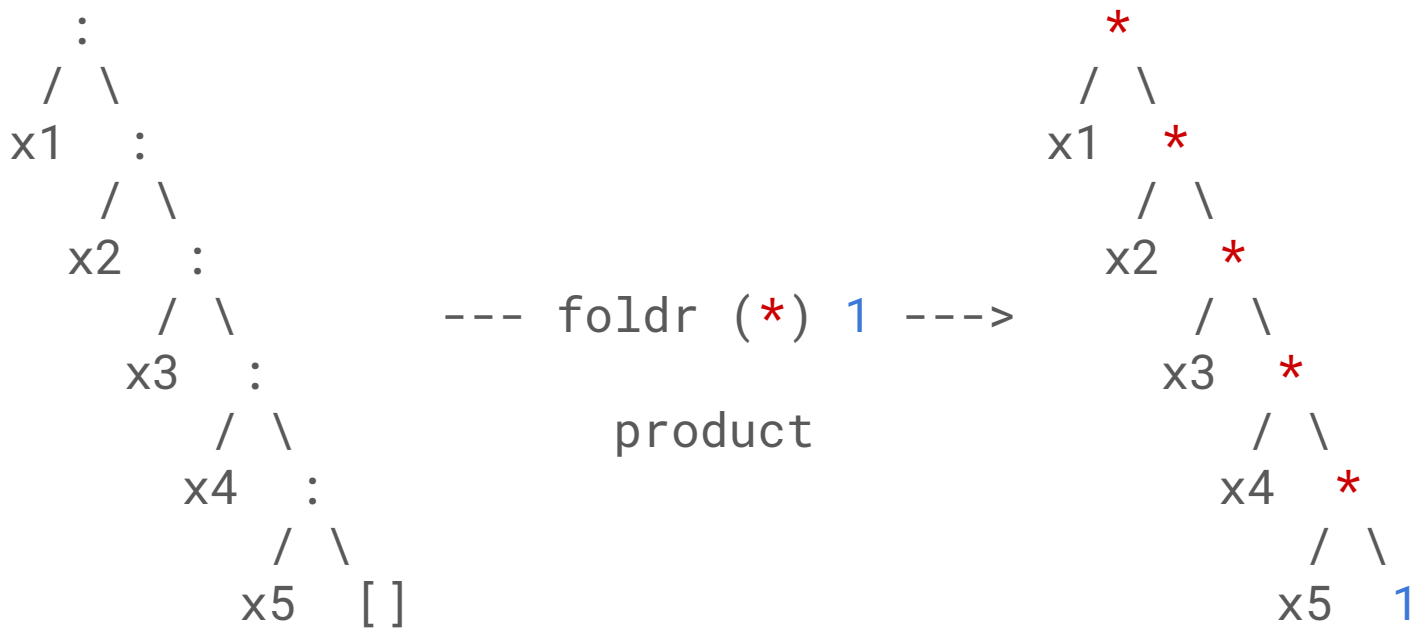


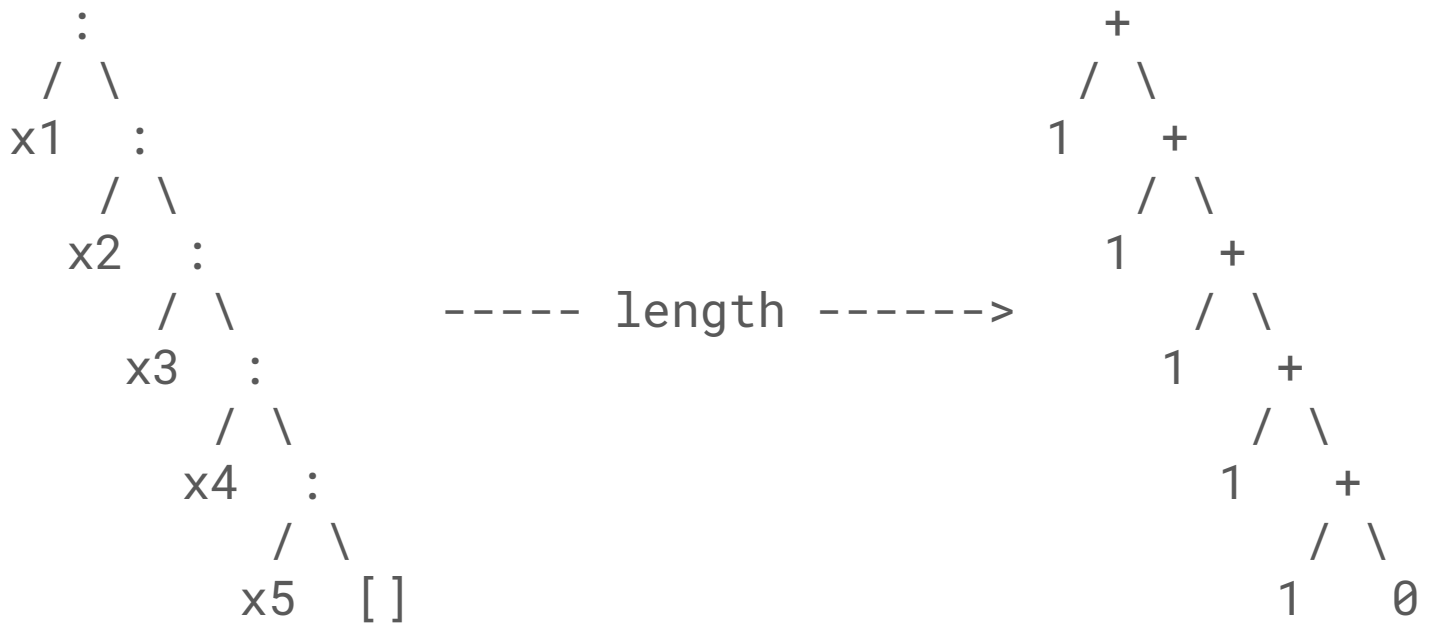
10

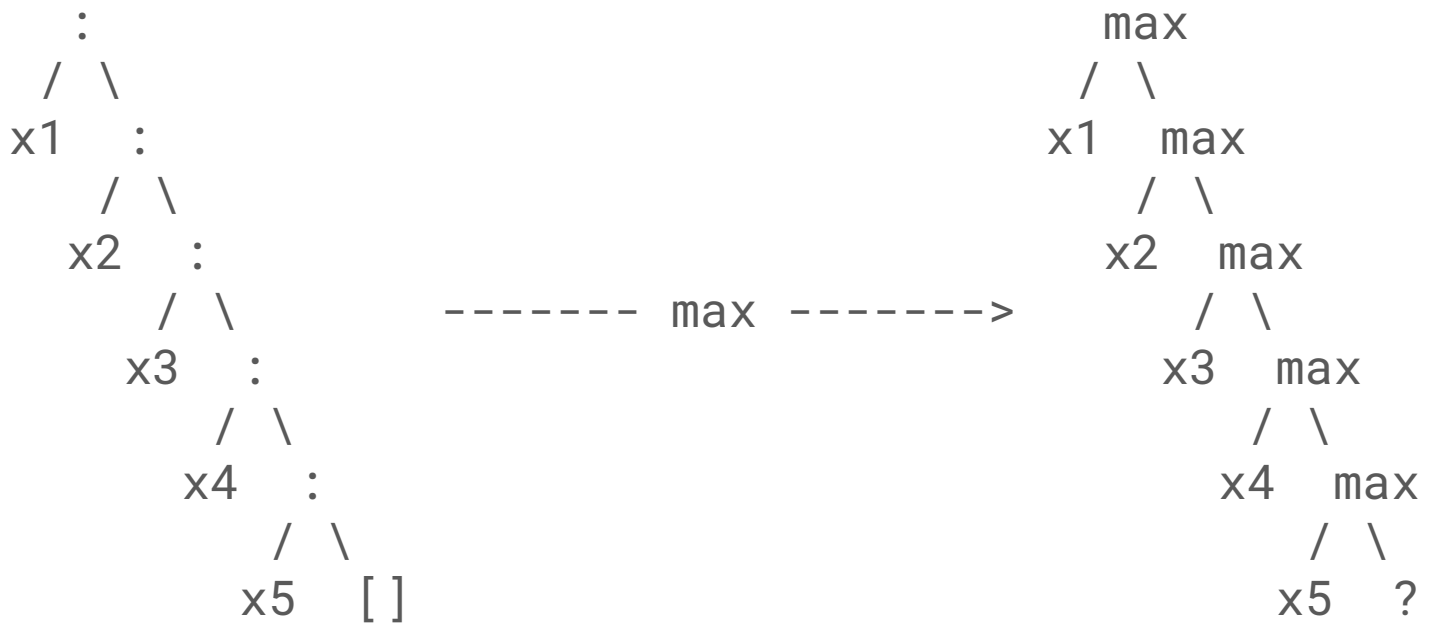
```
foldr (+) 0 [1, 2, 3, 4]
```


:
x1
:
x2
:
x3
:
x4
:
x5
[]
----- foldr f z --->
f
x1
f
x2
f
x3
f
x4
f
x5
z


:
x1 :
x2 :
x3 :
x4 :
x5 []
--- foldr (+) 0 --->
sum
+
x1 +
x2 +
x3 +
x4 +
x5 0


:
x1
:
x2
:
x3
:
x4
:
x5
[]
--- foldr (*) 1 --->
product
*
x1
*
x2
*
x3
*
x4
*
x5
1


:
x1
:
x2
:
x3
:
x4
:
x5
[]
----- length ------>
+
1
+
1
+
1
+
1
0


:
x1 :
x2 :
x3 :
x4 :
x5 []
------- max ------->
max
x1 max
x2 max
x3 max
x4 max
x5 ?

end